

Mechanically Verified Optimization and UPPAAL Export for Timed Büchi Automata

ELIE FARES¹, JEAN-PAUL BODEVEIX², HUSSAIN AL-AQRABI¹ AND AZAR SALAMI¹

¹Higher Colleges of Technology

Sharjah, United Arab Emirates

(e-mail: efares@hct.ac.ae, halaqrabi@hct.ac.ae, asalami@hct.ac.ae)

²IRIT, UPS, Université de Toulouse, France (e-mail: bodeveix@irit.fr)

ABSTRACT Timed Büchi automata (TBAs) produced by translators or supplied by other tools can contain redundant timing constraints, resets, transitions, clocks, and states, and may use disjunctive invariants that are unsuitable for direct UPPAAL export. We present a pipeline mechanically verified in Rocq (formerly Coq) that synthesizes location invariants, propagates timing bounds, removes infeasible transitions and dead resets, merges synchronously reset clocks and equivalent transitions and locations, and converts the result to conjunctive guards and invariants. Every pass preserves the timed-word language of every input TBA under the development's existential initial-clock semantics. In particular, the optimization theorem is generic in the automaton and in the number of rounds; the export theorem composes the verified transformations into a UPPAAL-oriented representation. The transformations are extracted to OCaml and used in `mtl2tba`. We report measurements on 31 automata generated from a fixed Metric Temporal Logic (MTL) benchmark, plus UPPAAL case studies. The data show large transition reductions on several inputs, while export can increase the edge count when it removes disjunctive invariants. These experiments evaluate the end-to-end tool on the benchmark automata; they do not measure a broad corpus of arbitrary TBAs.

INDEX TERMS Rocq/Coq, formal verification, program extraction, timed automata, timed Büchi automata, UPPAAL, verified optimization.

I. INTRODUCTION

Timed Büchi automata are a common intermediate representation for real-time requirements and are also used as inputs to timed-language analysis tools. An automaton that is correct at the language level may still be inconvenient for downstream verification: its transition guards can repeat constraints, some resets may never affect a future guard, multiple clocks may evolve in lockstep, and the same behavior may be represented by many transitions. In addition, merging transitions can produce disjunctive guards and invariants, while UPPAAL-oriented automata are most useful with conjunctive guards and conjunctive upper-bound invariants.

An optimization pass is safe only when it preserves accepted timed words, including the treatment of clock resets, event labels, and Büchi acceptance. Several passes also interact. Propagating a lower bound may make a transition contradictory, but a synthetic nonnegativity guard can keep an otherwise dead clock live unless guards are normalized before reset analysis. Splitting a disjunctive invariant into copies also requires a condition on entry: choosing a copy whose invariant admits less time than another copy can change the accepted language.

This paper presents a mechanically verified post-processing and export pipeline for TBAs. Its central results

quantify over an arbitrary input TBA, rather than only automata generated from a particular temporal logic. In Coq, we prove that each optimization and export operation preserves the automaton's timed-word language under the acceptance semantics formalized in the development. The optimizer theorem holds for every finite number of rounds, so the implementation can stop after reaching a structural fixed point or after its configured iteration bound without relying on convergence for correctness.

The transformations have two stages. The optimization stage synthesizes upper-bound location invariants, propagates target invariants backward into guards, propagates entry lower bounds forward, removes contradictory and unreachable transitions, removes dead resets, merges clocks reset synchronously, tightens guards, and merges transitions and locations. The export stage merges transitions with equal source, target, label, and resets; eliminates disjunctive invariants by creating copies with conjunctive invariants and clock-difference entry conditions; splits disjunctive guards; normalizes the result; and emits symbolic transitions for drawing.

The verified transformations are extracted to OCaml and used by `mtl2tba`. That tool includes an upstream MTL-to-TBA construction and an interface to Spot, but the preservation theorems in this paper apply to every TBA passed to

the verified post-processing functions. The experiments use automata from 31 fixed MTL benchmark configurations, not a diverse arbitrary-TBA corpus. We therefore distinguish the universal proof claims from the benchmark-specific performance measurements.

The contributions are:

- a Coq development of language-preserving TBA transformations, including invariant synthesis and propagation, clock and state simplification, transition merging, and disjunctive-invariant elimination;
- a generic theorem for every TBA and every number of optimization rounds, plus a verified export composition whose final guards and invariants are conjunctive;
- an extracted implementation and an explicit account of its interface and trust boundaries; and
- a reproducible evaluation on the tracked 31-configuration benchmark and four system-model case studies, with results and limitations reported from repository data.

The integrated tool reuses the same shared-clock Rocq MTL-to-TBA translation proof presented in the companion TSE manuscript as its upstream construction. This translation proof is a prerequisite and is not a new contribution of this paper. The new formal results here begin with generic TBA optimization and export, and apply to every TBA passed to the verified post-processing functions. End-to-end MTL correctness additionally composes with the upstream translation result and its LTL-to-Buchi backend contract.

II. TIMED AUTOMATA AND THE PRESERVATION CONTRACT

A. TIMED WORDS AND TBAS

A timed word is an infinite sequence $w = (a_i, t_i)_{i \in \mathbb{N}}$, where a_i is an event from a finite alphabet Σ , timestamps are strictly increasing, and time diverges: $\lim_{i \rightarrow \infty} t_i = +\infty$. For each position i , let $\delta_i = t_{i+1} - t_i$ be the time from event i to the next event. A clock valuation maps a finite clock set X to $\mathbb{R}_{\geq 0}$. Guards are finite conjunctions of event literals and clock comparisons $x \bowtie c$ for $\bowtie \in \{<, \leq, =, \geq, >\}$; transitions also carry a set of clocks to reset.

A TBA is a tuple $\mathcal{A} = (L, \ell_0, X, E, Acc)$, where L is a finite location set, ℓ_0 is the initial location, X is the clock set, E is a finite set of transitions $(\ell, \alpha, g, Z, \ell')$, α is an event condition, g is a clock guard, $Z \subseteq X$ is a reset set, and $Acc \subseteq L$ is the set of Büchi-accepting locations. The starting automata in the verified optimization are represented without location invariants; synthesis adds them as a separate annotation. Exported invariants are conjunctions of upper bounds.

The Coq development interprets transitions over an extended timed word that records clock values and reset decisions. Here v_i is the clock valuation at event i , and Z_i records the clocks reset by the transition on that event. Since a reset occurs at t_i , the next-event valuation includes the elapsed

interval δ_i :

$$v_{i+1}(x) = \begin{cases} \delta_i, & x \in Z_i, \\ v_i(x) + \delta_i, & x \notin Z_i. \end{cases}$$

A run chooses one enabled transition per position and visits accepting locations infinitely often. The accepted language existentially quantifies over clock-consistent extensions of the base timed word, including the initial clock values. This is the acceptance notion of the generic preservation theorems; conventional initialization at zero is a separate deployment concern.

For an annotated automaton, an invariant must hold throughout the delay spent at its location. Since synthesized invariants are conjunctions of non-strict upper bounds and clocks are nondecreasing during a delay, holding at the event at the end of the delay is equivalent to holding throughout that stay. The proof formalizes this trace-model equivalence and uses it to show that the annotations and strengthening operations preserve timed words.

B. WHAT EXPORT MUST PRESERVE

The export must retain the same transition alphabet, Büchi marking, clock meaning, and timed-word language. It may change the number of locations, transitions, and clocks, and may introduce difference constraints in transition guards to choose among invariant copies. Such choices must not create or delete an accepted timed word. The target representation has conjunctive guards and conjunctive location invariants; a later symbolic representation groups transitions with identical source, target, and reset sets under Boolean labels.

All main preservation theorems below are stated for arbitrary input TBAs. Their proof does not require properties of the MTL translation or Spot. An upstream correctness theorem can be composed with these results, but Spot's correctness and the soundness of the front end are outside the generic pass theorems.

III. VERIFIED OPTIMIZATION

After invariant synthesis, the optimization pipeline alternates timing propagation with simplification. Synthesized upper bounds expose how long an automaton can remain at a location. Propagating them to incoming guards can make lower and upper constraints jointly inconsistent; forward propagation supplies complementary lower-bound information. The pipeline then removes transitions, resets, clocks, and duplicate structure that no longer affect the language.

A. LOCATION INVARIANTS ON THE TRACE MODEL

A location invariant maps every clock x either to an upper bound $M \in \mathbb{R}$, standing for the constraint $x \leq M$, or to $+\infty$ (no constraint). We interpret automata with invariants on the extended words of Section II, as we do for automata without invariants. Along a run r , the automaton stays in location $r(i)$ between the events $i - 1$ and i , and the invariant of $r(i)$ must hold during the whole stay, that is, at the clock

TABLE 1. Verified post-processing passes. Each preservation theorem quantifies over every input TBA under the Coq timed-word semantics. Detailed definitions and proof arguments follow in this section.

Pass	Coq preservation theorem
Upper-bound invariant synthesis	add_invariants_accepts
Backward propagation (n steps)	propagate_n_accepts
Forward propagation of lower bounds	forward_accepts
Two-sided invariants	add_two_sided_invariants_accepts
Removal of contradictory transitions	remove_contradictory_accepts
Removal of locations without entering transitions	remove_unreachable_accepts
Removal of dead resets	remove_dead_resets_accepts
Merging synchronously reset clocks	merge_all_accepts
Normalization of guards	normalize_accepts
Removal of unsatisfiable action labels	remove_unsat_accepts
Merging transitions	merge_trans_accepts
Merging equivalent locations	merge_states_all_accepts
Iterated pipeline (n rounds)	optimize_accepts
Merging transitions for export	merge_transitions_accepts
Elimination of disjunctive invariants	split_accepts
Splitting of disjunctive guards	explode_accepts
Normalization of exported guards	normalize_d_accepts
Removal of copies never entered	dprune_n_accepts
Merging of the initial location	dmerge_states_all_accepts
Symbolic transitions	symbolic_ext_accepts
Complete export	export_accepts

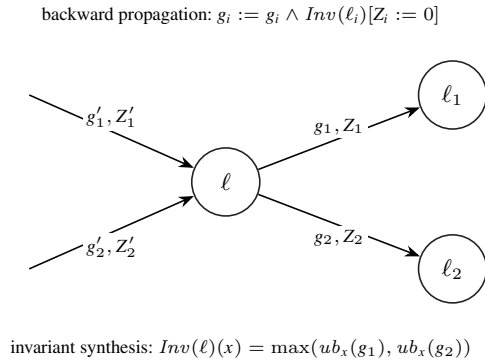


FIGURE 1. Invariant computation. Successor invariants strengthen outgoing guards; upper bounds of the resulting guards resynthesize the source invariant. Reset sets determine which target-clock constraints become constant.

values $\text{val}_\rho(i, x) - \tau$ for every delay $\tau \in [0, \Delta_{i-1}]$, where $\Delta_{i-1} = t_i - t_{i-1}$ (and at the values $\text{val}_\rho(0, x)$ for the initial location). This is the condition of Section II, read at the event that ends the stay. Since clocks increase while time elapses, an invariant made of upper bounds holds during the stay if and only if it holds at the event that ends it (lemma `inv_during_iff_at_event`). For the two-sided invariants of Section III-C, a lower bound $x \geq m$ must likewise hold during the whole stay, hence at its beginning.

B. INVARIANT SYNTHESIS AND BACKWARD PROPAGATION

In the exported timed automaton, upper-bounded guards need invariants so that time does not elapse past all useful outgoing transitions. We compute location invariants from outgoing

upper bounds and propagate successor requirements backward, using location invariants to constrain admissible time elapse as in standard UPPAAL models [1].

a: Upper bound of a guard.

For a conjunctive guard g and a clock x , let $\text{ub}_x(g)$ be the smallest bound m of the constraints $x \leq m$, $x < m$, and $x = m$ occurring in g , and $+\infty$ if g contains none. Other constraints are ignored. A valuation satisfying g satisfies $x \leq \text{ub}_x(g)$.

b: Invariant synthesis.

For a location ℓ with outgoing transitions $\ell \xrightarrow{g_i, Z_i} \ell_i$, $i = 1, \dots, m$, the synthesized invariant is

$$\text{Inv}(\ell)(x) = \max_{1 \leq i \leq m} \text{ub}_x(g_i),$$

with $\text{Inv}(\ell)(x) = +\infty$ when ℓ has no outgoing transition. Thus a clock that some outgoing guard does not bound does not appear in the invariant. The bound is always non-strict, even when it comes from a strict guard $x < m$: the invariant only needs to be implied by the guard of the next transition, and strictness is kept in the guards themselves.

Theorem III.1 (Invariant synthesis). *For every TBA $\mathcal{A}$, the automaton $\mathcal{A}$ with the synthesized invariants accepts the same timed words as $\mathcal{A}$.*

Proof. Adding invariants can only remove runs. Conversely, let r be an accepting run on ρ . At every event i , the transition taken from $r(i)$ is enabled, so its guard holds at the values $\text{val}_\rho(i, x)$, which are therefore at most $\text{Inv}(r(i))(x)$. By the lemma `inv_during_iff_at_event`, the invariant of $r(i)$ holds during the whole stay. (Coq: `add_invariants_correct`.) $\square$

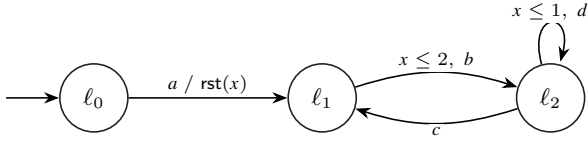


FIGURE 2. Automaton used to illustrate invariant propagation, with one clock x and actions a – d . Edges are labeled “guard, action / resets” (the guard is omitted when it is $\top$); only the edge $\ell_0 \rightarrow \ell_1$ resets x . The incoming arrow marks the initial location; acceptance plays no role here. The computed fixed point is $Inv(\ell_1) = Inv(\ell_2) = (x \leq 2)$.

c: Backward propagation.

One propagation step strengthens every transition $\ell \xrightarrow{g, Z} \ell'$ with the invariant of its target after the resets:

$$g := g \wedge Inv(\ell')[Z := 0],$$

where $Inv(\ell')[Z := 0]$ replaces every bound $x \leq M$ on a reset clock $x \in Z$ by the constant constraint $0 \leq M$, that is, by $\top$ if $M \geq 0$ and by $\perp$ otherwise. Invariants are then synthesized again from the strengthened guards.

Theorem III.2 (Backward propagation). *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, the automaton obtained by n propagation steps accepts the same timed words as $\mathcal{A}$.*

Proof. Strengthening guards can only remove runs. Conversely, consider an accepting run that takes $\ell \xrightarrow{g, Z} \ell'$ at event i and then a transition $\ell' \xrightarrow{g', Z'} \ell''$ at event $i + 1$. The second guard holds at event $i + 1$, so the values at that event are bounded by $Inv(\ell')$. Clock consistency gives $\text{val}_\rho(i + 1, x) = \Delta_i$ for $x \in Z$ and $\text{val}_\rho(i + 1, x) = \text{val}_\rho(i, x) + \Delta_i \geq \text{val}_\rho(i, x)$ otherwise; hence the strengthened guard $g \wedge Inv(\ell')[Z := 0]$ holds at event i . The argument uses clock consistency, so the theorem is stated on timed words. (Coq: `propagate_accepts` and `propagate_n_accepts`.) $\square$

Each round of the optimization pipeline of Section III-H applies one propagation step. The tool repeats the rounds until the automaton is structurally equal to the previous one, as an OCaml value, or until 50 rounds have been performed; the bound guarantees termination, and the theorem holds for every number of steps, whether or not a fixed point is reached. On our benchmarks, the rounds reach a fixed point after at most five rounds. The invariant computation at one location is summarized in the article’s Figure 1.

d: Example.

We illustrate the algorithm on the automaton of Fig. 2, which contains a two-location cycle $\ell_1 \rightarrow \ell_2 \rightarrow \ell_1$ and a self-loop at ℓ_2 . It has one clock x , the initial location ℓ_0 , and the transitions

$$\begin{aligned} \ell_0 &\xrightarrow{a, Z=\{x\}} \ell_1, & \ell_1 &\xrightarrow{b, x \leq 2, Z=\emptyset} \ell_2, \\ \ell_2 &\xrightarrow{c, \top, Z=\emptyset} \ell_1, & \ell_2 &\xrightarrow{d, x \leq 1, Z=\emptyset} \ell_2. \end{aligned}$$

The edge $\ell_1 \xrightarrow{b} \ell_2$ requires $x \leq 2$, and the self-loop at ℓ_2 requires the tighter bound $x \leq 1$.

- 1) *Synthesis.* $Inv(\ell_1)(x) = ub_x(x \leq 2) = 2$. At ℓ_2 , $Inv(\ell_2)(x) = \max(1, +\infty) = +\infty$, because the transition toward ℓ_1 does not bound x . At ℓ_0 , the only outgoing guard is $\top$, so $Inv(\ell_0)(x) = +\infty$.
- 2) *Propagation.* The transition $\ell_2 \xrightarrow{c, Z=\emptyset} \ell_1$ becomes guarded by $\top \wedge Inv(\ell_1) = (x \leq 2)$. The transition $\ell_0 \xrightarrow{a, Z=\{x\}} \ell_1$ receives $(0 \leq 2) \equiv \top$ and is unchanged.
- 3) *Synthesis.* At ℓ_2 , the outgoing guards are now $x \leq 1$ and $x \leq 2$, so $Inv(\ell_2)(x) = \max(1, 2) = 2$.
- 4) *Propagation.* The transition $\ell_1 \xrightarrow{b, x \leq 2} \ell_2$ receives $x \leq 2$, which it already contains, and the self-loop at ℓ_2 receives $x \leq 2$, which is implied by $x \leq 1$. Nothing changes.

The fixed point is $Inv(\ell_0) = \top$ and $Inv(\ell_1) = Inv(\ell_2) = (x \leq 2)$. While the automaton remains in the cycle $\ell_1 \leftrightarrow \ell_2$, the clock x cannot exceed 2, and at every reachable valuation at least one outgoing transition of the current location is enabled. The invariants do not remove every blocking behavior: since x is never reset in the cycle, time is blocked once $x = 2$. They make the upper bounds of the property automaton explicit, and Theorems III.1 and III.2 guarantee that they do not change its language.

C. FORWARD PROPAGATION OF LOWER BOUNDS AND SIMPLIFICATION

a: Entry bounds.

Dually, for a transition $t = \ell \xrightarrow{g, Z} \ell'$ and a clock x , the *entry bound* of x along t is 0 if $x \in Z$ and otherwise the largest bound $lb_x(g)$ of the constraints $x \geq m$, $x > m$, and $x = m$ of g , if any. The entry bound of x at a location ℓ' is the minimum of the entry bounds of x along the transitions entering ℓ' ; for the initial location, the initial entry, with bound 0, also counts. If some entering transition gives no bound on x , the clock x has no entry bound at ℓ' . Since clocks are nonnegative and do not decrease while time elapses, every clock with entry bound m at ℓ' satisfies $x \geq m$ during every stay in ℓ' .

b: Forward propagation.

The forward step adds $x \geq m$ to the guard of every transition leaving ℓ' , for every clock x with entry bound m at ℓ' (Coq: `forward_accepts`). The entry bounds can also be read as lower-bound invariants: the two-sided automaton that carries the synthesized upper-bound invariants and the entry bounds as lower-bound invariants accepts the same timed words (`add_two_sided_invariants_accepts`). UPPAAL rejects lower-bound invariants, and their only role is to make infeasible transitions visible: after forward and backward propagation, a transition whose source location bounds a clock from below and whose target bounds it from above carries both bounds in its guard.

c: Simplification.

Three simplifications follow. First, a transition whose guard contains contradictory constraints is removed: a lower bound and an upper bound on the same clock with $u < l$, or with

$u = l$ and one of them strict, or an upper bound $x \leq u$ with $u < 0$ or $x < u$ with $u \leq 0$. Such a transition is never enabled (Coq: `remove_contradictory_accepts`). Second, the transitions leaving a non-initial location that has no entering transition are removed: such a location never occurs on a run after position 0 (Coq: `remove_unreachable_accepts`). The iteration of the pipeline removes chains of such locations one step at a time. A location whose only entering transitions are self-loops keeps them, but it is unreachable from the initial location; the tool counts and prints only the locations reachable from the initial one and their transitions. Third, a transition whose action label is unsatisfiable, because it requires two different actions or an action and its negation, is removed (Coq: `remove_unsat_accepts`). Such labels arise because the translator treats an action a and the atom $\neg a$ of $\mathcal{T}(f)$ as independent propositions; removing them also enables clock merging, since the resets of a transition that can never be taken would otherwise prevent it.

D. DEAD RESETS AND MERGING OF SYNCHRONOUSLY RESET CLOCKS

a: Dead resets.

A clock x is *live* at a location ℓ if some transition that tests x in its guard can be taken from ℓ , possibly after other transitions. The live locations are computed as a backward closure, along all transitions, from the sources of the transitions that test x ; the other locations are *dead* for x . Since the correctness of the pass does not depend on how this closure is computed, the pass checks the two properties that the proof uses, namely that no transition leads from a dead location to a live one and that none tests x from a dead location, and then removes x from the reset sets of the transitions that enter a dead location. After such a transition, the value of x is never read again, so whether it is reset does not matter. The proof builds, from every accepted extended word, an extended word that differs only in the reset decisions of x , and conversely (Coq: `remove_dead_resets_clock_accepts`). The pass is applied to every clock (`remove_dead_resets_accepts`).

b: Synchronous clocks.

The verified translation shares the clock of syntactically identical timed subformulas. Clocks of *different* subformulas can also be merged when they are reset synchronously. Two clocks $x \neq y$ are *mergeable* if

- (i) every transition resets x if and only if it resets y ;
- (ii) no transition enters the initial location; and
- (iii) every transition leaving the initial location resets x and does not test it.

Then, after the first event, x and y always have the same value, and every test of x can be replaced by the same test on y (Coq: `merge_accepts`). The condition is checked by a Boolean function proved to imply it, and the pass tries all pairs of reset clocks (`merge_all_accepts`). The merged clock x is no longer tested, so its remaining resets are removed by the dead-reset pass in the next round. This verified merging is restricted

to synchronously reset clocks; merging clocks whose values coincide for other reasons is not covered.

E. NORMALIZATION OF GUARDS

Backward and forward propagation add constraints to guards. Every time a guard is strengthened, it is *tightened*: a constraint implied by another constraint of the same guard on the same clock is dropped, so that a guard keeps at most one lower bound and one upper bound per clock (an equality counting as both); for instance, $x \leq 5 \wedge x < 2$ becomes $x < 2$. The implication test compares bounds and strictness, and is proved sound (Coq: `tighten_holds`). Normalization tightens every guard and also removes the lower bounds $x \geq m$ with $m \leq 0$ and $x > m$ with $m < 0$, which every clock value satisfies; it is needed after the merging of clocks, which renames constraints. The removal of trivial lower bounds relies on the nonnegativity of clock values, so the theorem is stated on timed words (Coq: `normalize_accepts`).

F. MERGING OF TRANSITIONS AND OF LOCATIONS

a: Transitions.

Two transitions with the same source, target, and resets are merged when the union of their enabling conditions is again a conjunction. A transition is removed when another transition with the same source, target, and resets is *weaker*, that is, when its label and guard imply those of the other transition, constraint by constraint (subsumption). Two transitions whose labels, or guards, differ only by a literal and its negation, such as p and $\neg p$, $x < 2$ and $x \geq 2$, or $x \leq 5$ and $x > 5$, are replaced by one transition without that literal (resolution). The pass preserves the language because every enabled transition has an enabled counterpart with the same source and target, and conversely (Coq: `merge_trans_accepts`). The other unions are disjunctive; they are formed by the export (Section III-G).

b: Locations.

A location ℓ' is merged into a location ℓ when both are accepting or both are not, and when every transition leaving ℓ' has a twin leaving ℓ , and conversely: same label, guard, and resets, compared as sets, and same target once ℓ' is replaced by ℓ . The transitions entering ℓ' are redirected to ℓ , and those leaving ℓ' are removed. A run of the original automaton becomes a run of the merged one by replacing ℓ' by ℓ ; conversely, a run of the merged automaton is followed in the original one, taking in ℓ' the twin of the transition taken in ℓ (Coq: `merge_states_all_accepts`; “state” in a Coq name means location). The condition on the Büchi marking is necessary: merging an accepting location with a non-accepting twin would accept runs that visit only the latter infinitely often.

G. TRANSITION MERGING AND ELIMINATION OF DISJUNCTIVE INVARIANTS

The export works on a model whose guards are in disjunctive normal form over clock constraints and clock-difference

constraints $x - y \leq b$, and whose location invariants are disjunctions of conjunctions of upper bounds. It is interpreted on the same trace model as before. The export merges transitions, splits locations, splits disjunctive guards, normalizes the result, and merges the initial location with an equivalent one, each pass being proved to preserve the language of timed words; it ends with an automaton whose guards and invariants are conjunctive, as UPPAAL requires (Coq: `export_accepts`, `export_guards_conjunctive`, and `export_invariants_conjunctive`).

a: Transition merging.

Transitions with the same source, target, action label, and reset set are merged into one transition whose guard is the disjunction of their guards (Coq: `merge_transitions_accepts`). The invariant of a location ℓ is then naturally disjunctive: it has one disjunct $ub(g)$, the conjunction of the upper bounds of g , for every conjunct g of every guard leaving ℓ :

$$I = I_1 \vee \dots \vee I_r.$$

A disjunct that implies another disjunct is redundant and is dropped. The resulting invariant is more precise than the clock-by-clock maximum of Section III-B, which is its conjunctive over-approximation.

b: Splitting locations.

UPPAAL does not allow disjunctive location invariants: invariants must be conjunctions of clock constraints defining convex, past-closed zones. A location with invariant $I_1 \vee \dots \vee I_r$ is therefore replaced by locations with conjunctive invariants and the same timed behavior. Bodeveix and Filali studied disjunctive invariants in UPPAAL [2]. Figure 3 in the article summarizes the transformation.

The location L is replaced by one copy L_k for each disjunct I_k , with the conjunctive invariant I_k . Every transition entering L is replicated toward every copy L_k , and its guard is strengthened with the entry condition Ent_k after the resets; every transition leaving L is replicated from every copy. The disjunctive invariant itself is never built: the copies introduce the conjunctive invariants directly, and the theorem compares the split automaton with the automaton without invariants. The run starts in a dedicated copy of the initial location, without an invariant, since it is not entered by a transition. In the Coq development, the copy k of location ℓ is numbered $\ell \cdot \kappa + k$, where $\kappa - 1$ is the largest number of disjuncts and the copy $\kappa - 1$ is the initial one; numbers that correspond to no disjunct are unreachable locations.

c: Entry condition.

The disjuncts need not be mutually exclusive, so simple replication would introduce nondeterministic choices and could select a copy that permits less time to elapse than another one. The entry condition therefore selects a copy with the *longest*

admissible delay. The synthesized invariants have non-strict bounds only (Section III-B); let

$$I_k = \bigwedge_{c \in C_k} c \leq m_c^k.$$

From a valuation v , I_k admits the delays up to $Del_k(v) = \min_{c \in C_k} (m_c^k - v(c))$, with $Del_k(v) = +\infty$ when C_k is empty. The entry condition of L_k requires I_k and $Del_l(v) \leq Del_k(v)$ for every other copy l . The comparison $Del_l \leq Del_k$ holds when some bound $c' \in C_l$ is reached no later than every bound $c \in C_k$, which is expressed with clock differences:

$$Ent_k \triangleq I_k \wedge \bigwedge_{l \neq k} \bigvee_{c' \in C_l} \bigwedge_{c \in C_k} (c - c' \leq m_c^k - m_{c'}^l).$$

When C_k is empty, the copy admits every delay and, since redundant disjuncts are removed beforehand, no other C_l is empty, so that the conjunction over l is true. Ties are allowed: when two copies admit the same delay, both entry conditions hold.

Theorem III.3 (Elimination of disjunctive invariants). *The split automaton accepts the same timed words as the automaton without invariants.*

Proof. Every copy L_k behaves as L with additional constraints, so the split automaton accepts no new word. Conversely, consider an accepting run that enters L with the valuation v (after the resets) and leaves it after a delay τ along a transition whose guard has a conjunct g . This conjunct holds at $v + \tau$, hence so does the disjunct $ub(g) = I_j$, and $Del_j(v) \geq \tau$. If I_j was dropped because it implies a kept disjunct, that disjunct admits at least the same delays, so we may assume that I_j is kept (Coq: `disj_inv_cover`). Choose a copy k whose delay $Del_k(v)$ is maximal (Coq: `best`). Its entry condition holds at v , and $Del_k(v) \geq Del_j(v) \geq \tau$, so I_k holds during the whole stay. The successor copies are chosen in the same way at every position. Clock-difference constraints are invariant under the elapse of time, so Ent_k can be checked on the valuation after the resets, and $Ent_k[Z := 0]$ on the valuation before them. (Coq: `split_accepts`.) $\square$

The substitution $[Z := 0]$ maps a difference $c - c' \leq b$ to $0 \leq b$ when both clocks are reset, to $c' \geq -b$ when only c is reset, and to $c \leq b$ when only c' is reset. The disjunctions over c' in Ent_k , and those created by transition merging, are finally eliminated by putting the guards into disjunctive normal form and splitting every transition into one copy per disjunct (Coq: `explode_accepts`). In the worst case, the number of copies is the product of the sizes of the disjunctions.

d: Final normalization.

The guards are normalized before and after the split (Coq: `normalize_d_accepts`): in every conjunction, constraints implied by another one are removed, as in Section III-E, together with the trivial differences $x - x \leq b$ with $b \geq 0$; conjunctions containing a difference $x - x \leq b$

with $b < 0$ are unsatisfiable and are removed; and a conjunction that implies another conjunction of the same guard is removed. Finally, the transitions leaving a location that is neither initial nor the target of a transition are removed, until nothing changes (Coq: `dprune_n_accepts`); this removes in particular the transitions of the copies that correspond to no disjunct. The dedicated copy of the initial location is then merged into another location when both have the same acceptance status, equivalent invariants (an invariant with an empty disjunct being equivalent to no invariant), and the same outgoing transitions, compared as sets after redirection; this is the case, for instance, when the initial location has no invariant (Coq: `dmerge_states_all_accepts`). The resulting automaton has conjunctive guards with difference constraints and conjunctive location invariants made of upper bounds.

e: Symbolic transitions.

Each transition of the exported automaton reads a cube: a conjunction of action literals and a conjunction of clock constraints. For the drawing and for the comparison with tools whose transitions carry Boolean formulas, the last pass merges all the transitions with the same source, target, and resets into one *symbolic transition*, labeled by a disjunction of cases $E_1 \wedge c_1 \vee \dots \vee E_k \wedge c_k$, where E_j is a set of events, either finite or the complement of a finite set, and c_j a conjunction of clock constraints; the cases with the same constraints are merged by union of their sets of events. A symbolic transition is enabled when the current event belongs to some E_j whose constraints c_j hold, and its resets match those of the extended word. The symbolic automaton accepts the same extended words as the exported one, hence the same timed words for both the existential and the conventional acceptance (Coq: `symbolic_ext_accepts`, `MTL_to_symbolic_correct_with`, and `MTL_to_symbolic_correct0_with`). The UPPAAL model keeps the conjunctive transitions, since a UPPAAL edge synchronizes on one channel and its clock guard must be a conjunction.

H. ITERATED OPTIMIZATION

In right-to-left function-composition order, one optimization round is

```
optimize_step = merge_states
                ◦ merge_trans ◦ normalize
                ◦ merge_clocks ◦ dead_resets
                ◦ normalize ◦ unreachable
                ◦ unsat_labels ◦ contradictory
                ◦ forward ◦ propagate.
```

The order is deliberate. Forward propagation can add $x \geq 0$ after a reset; the first normalization removes this tautology before dead-reset analysis, which would otherwise treat the clock as live. Each operation may enable later simplifications, so the driver repeats the round until the TBA is structurally

unchanged or 50 rounds have run. The theorem is independent of this stopping rule and holds for every n .

Theorem III.4 (Optimization). *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, `optimize( $n, \mathcal{A}$ )` accepts exactly the timed words accepted by $\mathcal{A}$.*

Proof. Every constituent operation preserves acceptance, as listed in Table 1. Their composition preserves acceptance, and induction on n proves the statement for every number of rounds. The mechanized result is `optimize_accepts`. $\square$

The extracted function also returns synthesized upper-bound invariants and entry bounds. The tool compares successive automata structurally and stops when they are equal or after 50 rounds. The theorem holds independently of whether iteration reaches a fixed point or uses all 50 rounds. In the tracked benchmark, observed rounds reach a fixed point within five rounds; this is a measured property of those inputs, not a termination assumption in the proof.

IV. UPPAAL-ORIENTED EXPORT

The optimized automaton has normalized conjunctive guards. Export first merges transitions that agree on source, target, event label, and resets, placing their guards into a disjunction. The location invariant is then represented by disjunctions of conjunctions of upper bounds. UPPAAL-oriented output requires each such invariant to be conjunctive.

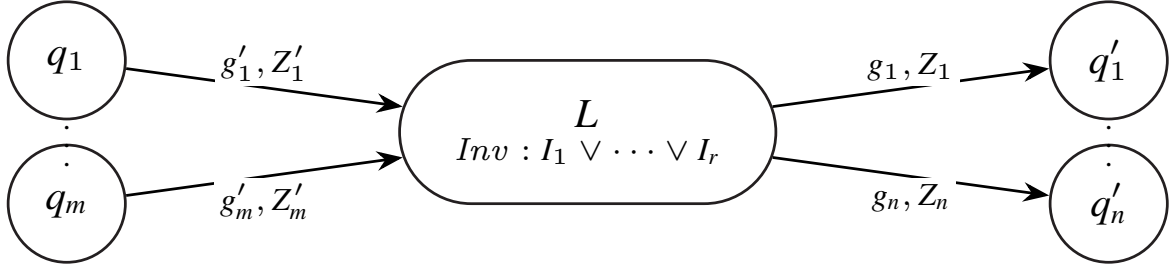
A location with invariant $I_1 \vee \dots \vee I_r$ is replaced by copies, one for each irredundant conjunctive component I_k . Incoming edges are replicated and strengthened with an entry condition; outgoing edges are replicated from each copy. The entry condition chooses a copy whose invariant admits a delay at least as long as each other copy would admit from the same valuation. This condition compares differences between clocks and is what introduces clock-difference guards in exported models. The initial location is handled by a dedicated copy because it has no incoming transition.

Figure 3 shows the split schematically. Replicating edges without the entry condition would permit a run to choose a copy with a shorter delay, potentially losing words. With the condition, the copies represent the original disjunction while preserving the longest admissible stay.

The export continues by splitting disjunctive guards into conjunctive transitions, normalizing, pruning unreachable copies, and merging the initial location when an equivalent copy exists. A final symbolic pass groups transitions with the same source, target, and resets and stores their labels as Boolean conditions for drawings.

Theorem IV.1 (Export preservation). *For every automaton accepted by the verified export interface, the complete export preserves its timed-word language. The resulting explicit automaton has conjunctive transition guards and conjunctive location invariants. Its symbolic representation also preserves acceptance.*

Original automaton



Transformed automaton

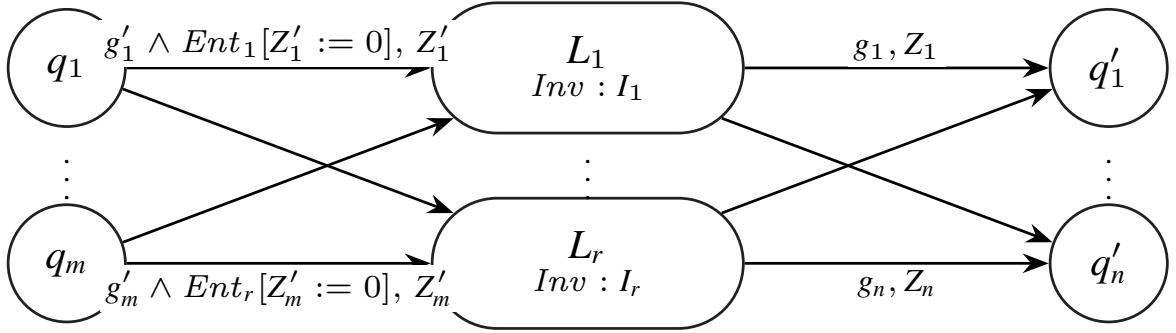


FIGURE 3. Elimination of a disjunctive invariant. Each copy has a conjunctive invariant, and the entry guard selects a copy that admits a maximal delay from the incoming valuation. The entry condition can contain clock-difference constraints.

Proof. Export composes verified transition merging, invariant splitting, guard splitting, normalization, pruning, initial-location merging, and symbolic grouping. Each component has a language-preservation theorem, including `split_accepts` for invariant elimination and `export_accepts` for the complete construction. Output shape is established by `export_guards_conjunctive` and `export_invariants_conjunctive`. $\square$

For reuse with another producer or checker, the theorem's output contract is summarized in Table 2. The complete construction and proof of the invariant split, including its entry-condition formula, appear in Section III-G. The theorem is generic in the automaton: it requires no MTL provenance, determinism, or particular Spot option.

A. GENERATED AUTOMATA WITH DISJUNCTIVE INVARIANTS

The following two tracked inputs show how the export handles disjunctive invariants in automata produced by the integrated tool. They illustrate the verified post-processing and export layer; the upstream translation is not part of the generic preservation claims in this paper.

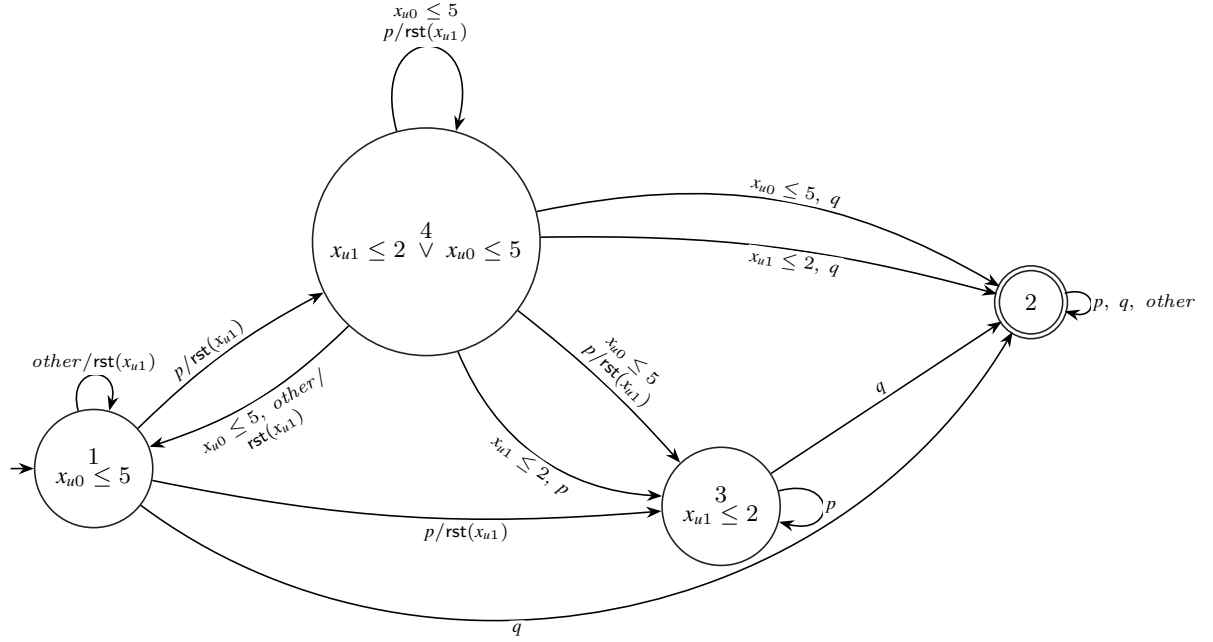
1) Nested upper-bounded until
For

$$\Diamond_{\leq 5}(p \cup_{\leq 2} q),$$

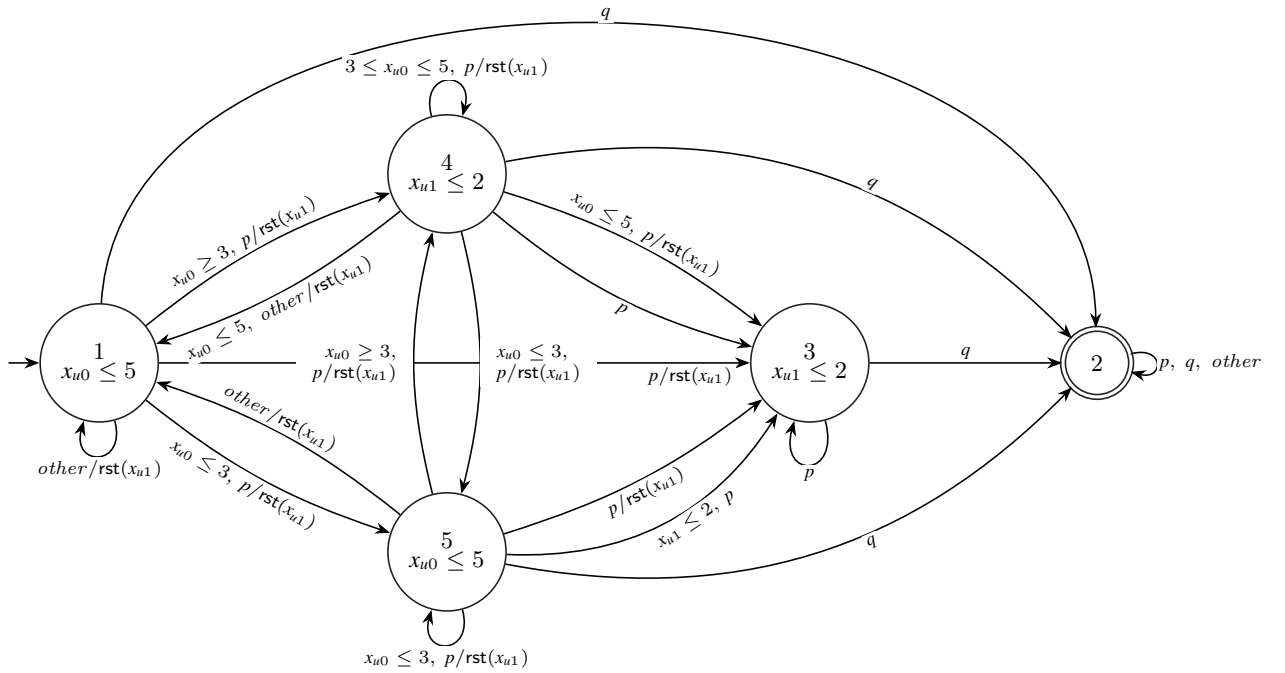
the optimized automaton has a disjunctive invariant at location 4, $x_{u1} \leq 2 \vee x_{u0} \leq 5$. Every edge entering that location resets x_{u1} . If the entry value of x_{u0} is v , the two invariant clauses permit delays of 2 and $5 - v$. The export splits location 4 into a copy with invariant $x_{u1} \leq 2$ and a copy with invariant $x_{u0} \leq 5$. Their entry conditions are respectively $x_{u0} \geq 3$ and $x_{u0} \leq 3$: each selects a copy that admits a maximal delay. Both are enabled at the tie $x_{u0} = 3$. Incoming transitions, including the self-loop, are replicated to both copies, and outgoing transitions are replicated from each.

Figure 4 shows the generated automaton before and after the split. For readability, the drawing omits guards implied by source invariants and edges subsumed by another edge with the same target, event, and resets; the exported automaton retains them. The output has 23 explicit transitions with `-init` (25 without it). This example uses Spot options `-B -D`; dead resets have already been removed.

At the timed-word level, the accepted behavior has a direct witness: either there are positions $j \leq k$ such that $t_j \leq 5, t_k -$



(a) Before invariant elimination.



(b) After invariant elimination.

FIGURE 4. Generated automaton for $\Diamond_{\leq 5}(p \cup_{\leq 2} q)$. Location invariants appear inside the nodes; edge labels are “guard, event / resets.” A comma-separated event list denotes separate transitions, and *other* denotes any event other than *p* and *q*. The double circle marks the accepting location. Both diagrams are after the initial `_init_` transition at time zero. Location 5 is created by the split. Guards implied by source invariants and subsumed edges are omitted from the drawing.

TABLE 2. Contract of the verified optimization and export pipeline. The explicit and symbolic outputs preserve the timed-word language of the TBA provided to the Coq interface.

Aspect	Guaranteed result
Timed-word language	Preserved by every optimization round and by complete export under the development's existential initial-clock semantics.
Explicit guards	Conjunctions of event literals and clock comparisons; export may introduce clock-difference comparisons in entry conditions.
Location invariants	Conjunctions of non-strict clock upper bounds.
Symbolic output	Groups transitions with the same source, target, and reset set under Boolean labels; the symbolic automaton also preserves acceptance.
Input adapter	A third-party parser or conversion into the Coq TBA representation requires a separate correspondence proof to join the verified boundary.

$t_j \leq 2$, and q holds at k , while p holds at every position i with $j \leq i < k$. The case $j = k$ is the immediate- q witness. The split preserves these same choices at location 4 while making each location invariant conjunctive.

2) Timed release obligation inside until

The second input is

$$\Diamond_{\leq 5}((\Box_{\leq 3}(p \vee q)) \cup_{\leq 2} q).$$

Its optimized automaton has seven locations before export and contains a disjunctive invariant (Fig. 5). The three distinct primitive timed subformulas use clocks x_{u0} , x_{u1} , and x_{r2} . Two syntactic occurrences of the bounded Release share x_{r2} because the clock key is the primitive formula.

When a q transition enters accepting location 5, the automaton permits only p or q until $x_{r2} > 3$, after which location 2 accepts every event. The bounded Release obligation can therefore remain pending after the Until witness. Two q transitions testing $x_{r2} > 3$ are unsatisfiable: at location 4, entry resets both relevant clocks; at location 3, entry resets x_{r2} and preserves x_{u1} with $x_{r2} \leq x_{u1} \leq 2$. The verified passes compare clock bounds with constants and do not track this clock-order relation, so the transitions remain in the optimized drawing. They cannot be taken. Export produces seven locations and 29 transitions.

3) Optimization and export of the F11 automaton

The F11 automaton for $(\Diamond_{\leq 5} p) \wedge (\Box_{< 2} \neg p)$ illustrates the interaction between the generic passes. Spot returns five states and eleven transitions. After one optimization round, contradictory action cubes have been removed, making location 2 unreachable; dead resets are removed, and clocks x and y are merged because every transition resets them synchronously. The second round removes the now-dead resets of y . The final UPPAAL-oriented automaton has four locations, seven transitions, one clock, and two location invariants. Figure 6 shows the state after the first round and the exported form. This example concerns the optimizer and export; its temporal translation is handled separately.

V. TOOL ARCHITECTURE AND USE IN UPPAAL

A. IMPLEMENTATION AND PROOF BOUNDARY

The OCaml module `optim.ml` is extracted from Coq. It implements the invariant and optimization functions, export passes, initialization check used by the upstream MTL workflow, and symbolic grouping. The surrounding `mtl2tba` executable contains a hand-written parser, calls Spot when building its upstream automaton, reads the returned LBTT automaton, drives the optimization loop, and prints UPPAAL XML and Graphviz DOT. In generic use, the verified functions operate on their Coq TBA type; a parser or converter for a third-party TBA format is an additional trusted interface unless verified separately.

Pass theorems are independent of the source of their TBA. End-to-end correctness for an MTL formula additionally composes with the upstream translation and its linear temporal logic (LTL)-to-Büchi contract. The Spot call, parser, LBTT reader, driver, and XML/DOT printers are outside the Coq pass proofs. The floating-point realization of extracted real-number operations is also trusted; the tool restricts bounds to integers in $[0, 2^{30}]$, for which inspected arithmetic is exact in the target representation. The repository records Rocq 9.0.1, OCaml 5.4.0, Spot 2.16, and UPPAAL 5.0.0 for the build and case-study artifacts.

The project-wide axiom `LTL_TO_BUCHI_CORRECT` states the assumed correctness of the upstream LTL-to-Büchi backend. It is used by the end-to-end theorem variants that do not take the backend contract as an explicit argument. The generic `optimize_accepts` and `export_accepts` theorems instead quantify over an arbitrary TBA and do not use that axiom; their proof boundary is the timed-word semantics of the input automaton.

In the integrated executable, Spot is called with state-based Büchi output, the small-automaton option, and LBTT transition labels. Clock predicates, event markers, and action literals are supplied as separate propositions, as required by the upstream translation contract. The hand-written reader retains positive and negative literals and expands each transition label into its cubes. The extracted functions then perform relaxation and reset completion, run the optimizer until the automaton stops changing or 50 rounds have elapsed, and apply the UPPAAL-oriented export. The proof is independent of this particular command and front end; the sequence describes

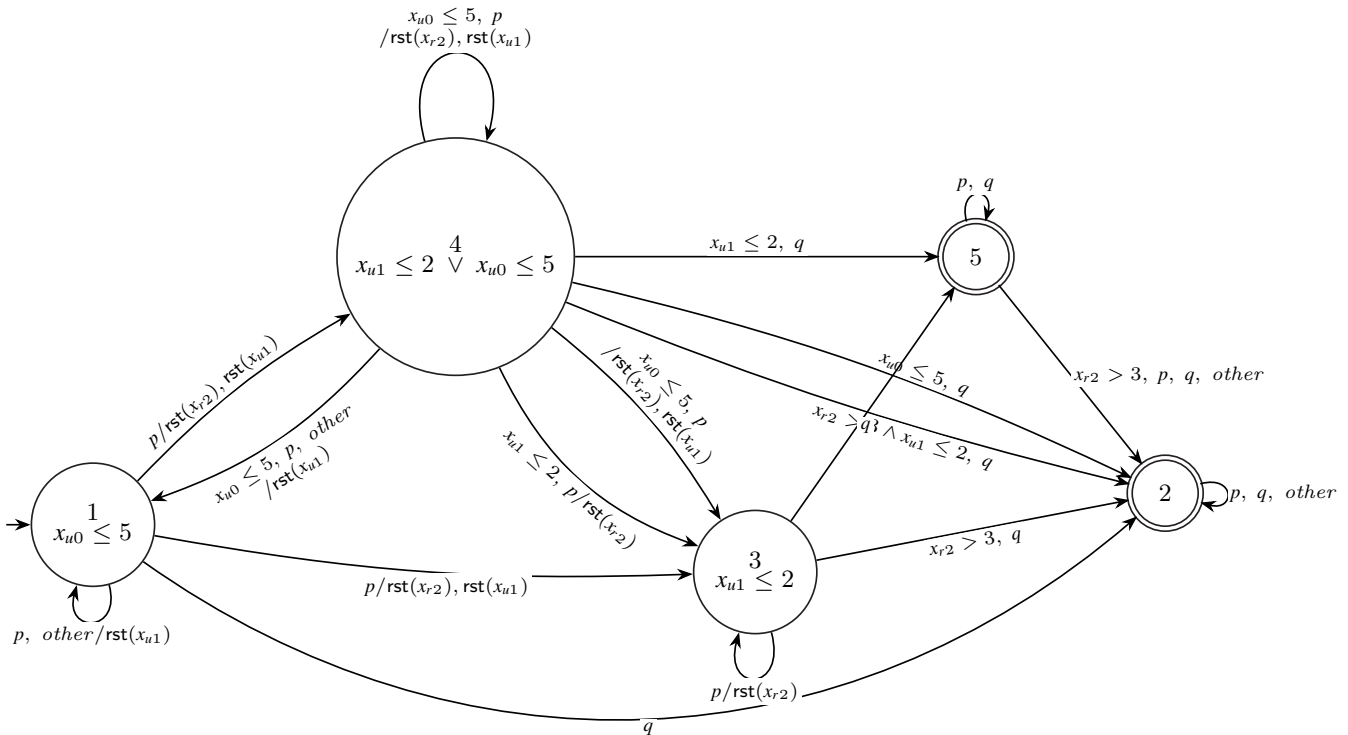


FIGURE 5. Partial drawing of the optimized automaton for $\diamond_{\leq 5}((\Box_{\leq 3}(p \vee q)) \cup_{\leq 2} q)$ before disjunctive-invariant elimination. Locations 1–5 are shown; two of the automaton’s seven locations and their incident edges are omitted. Clocks x_{u0} , x_{u1} , and x_{r2} belong to the three distinct primitive timed subformulas. The diagram is after the initial `_init_` transition; guards implied by source invariants are omitted. A comma-separated event list denotes separate transitions. The q -edges from locations 4 and 3 with $x_{r2} > 3$ are unsatisfiable for the reasons given in the text.

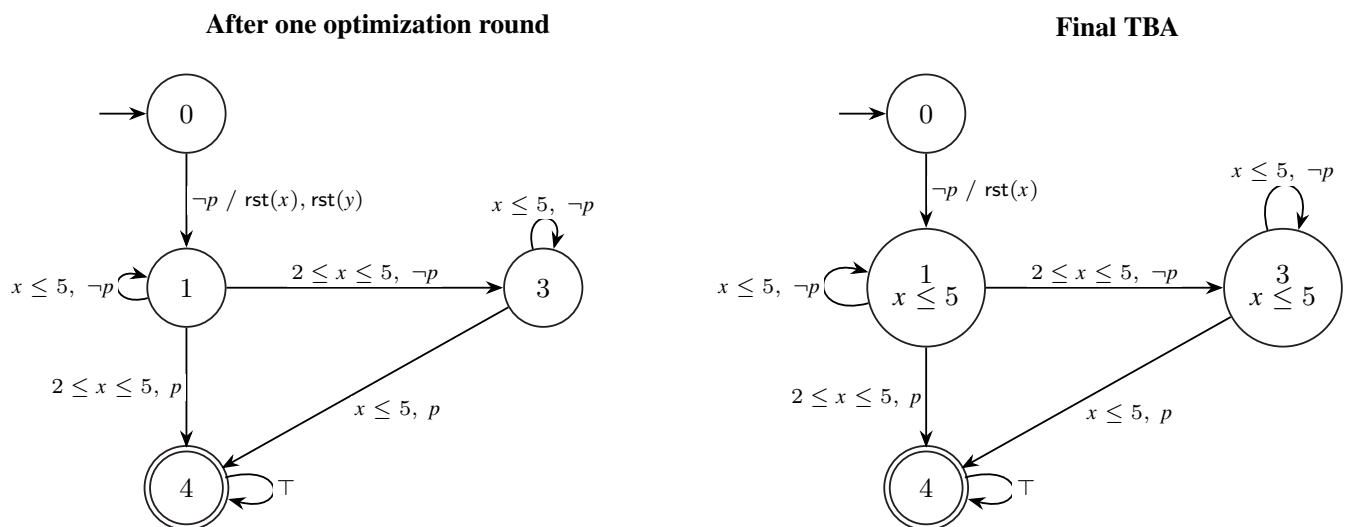


FIGURE 6. F11 after one optimization round and after UPPAAL-oriented export. The first panel shows the result of removing an unreachable location and the two synchronously reset clocks; the final panel shows the merged clock and synthesized invariants. Edges use the form “guard, action / resets,” and the double circle marks the accepting location.

how the generic verified layer is used in the available tool.

The generic theorems use existential initial clock values. UPPAAL starts clocks at zero. The verified `init_free` check and its zero-initialization theorem are part of the MTL-generated workflow; they do not provide an initialization

theorem for arbitrary third-party TBAs. Users should check the initialization contract of any external automaton.

B. INPUT CONTRACT FOR A GENERIC TBA

The generic interface starts from a finite TBA value (L, ℓ_0, X, E, Acc) in the Coq representation. Its transitions carry event conditions, finite conjunctions of unary clock comparisons, and reset sets; the starting value has no location-invariant annotation. The invariant synthesis pass derives that annotation from outgoing guards. The theorem does not require a complete or deterministic transition relation, a particular number of clocks, or a connection to a temporal-logic translator. A third-party producer can therefore use the proved functions after converting its automaton to this representation.

The conversion is a semantic boundary: it must retain the alphabet and the meaning of each event condition, preserve which clocks are tested and reset, and map accepting locations without changing their marking. The paper's repository provides the Coq type and extracted optimizer, but not a verified reader for arbitrary third-party formats. Any such reader belongs to the trusted interface until its correspondence with the Coq value is proved. The same applies to interpreting the result in a target checker: export proves language preservation for the Coq representation, while a separate bridge is needed to identify the checker's parsed model with that value. This boundary allows the central optimization theorem to remain generic while making the work needed to reuse it explicit.

C. INITIALIZATION INTERFACE

For the integrated MTL workflow, the verified `init_free` check establishes that the output language is unchanged when clocks start at zero. It succeeds on each completed default benchmark instance; R7 and R8 exceed the 300-second construction limit before producing an output. If the check fails, the driver can use `-init`, which inserts a single `_init_` event at time zero and rewrites the formula so that ordinary top-level timing bounds use that temporal origin. The system model must emit this event exactly once before its other events. This hand-written transformation is argued in the artifact but is not part of the generic optimization or export theorem. On the tracked benchmark, `-init` keeps the clock count unchanged, adds at most one location, and changes the transition count by between twelve fewer and one more than the default output. These are measured properties of the integrated workflow.

D. USING EXPORTED AUTOMATA IN UPPAAL

The language-preservation theorem concerns infinite timed words with one event at each position, strictly increasing timestamps, divergent time, and the development's clock initialization semantics. A UPPAAL composition must preserve the assumptions needed to interpret its traces in that model:

- (C1) *Events and time.* The system must synchronize each observable event with the property automaton, prevent zero-delay consecutive events (for example, reset a system clock z on each emission and require $z > 0$ on the next), and admit time-divergent infinite executions. Reachability alone does not rule out a Zeno continuation.

- (C2) *Initial clocks.* The generic Coq preservation theorems existentially quantify over initial clock values. UPPAAL initializes clocks to zero. The tool's verified `init_free` check supplies a zero-initialization result for the MTL-generated workflow when it succeeds; it is not a generic guarantee for a third-party TBA.
- (C3) *Acceptance.* UPPAAL's ordinary reachability interface does not check Büchi acceptance. For safety, use a violation automaton with an accepting sink that is reached when a violation occurs and has a loop on every event. Liveness requires a Büchi emptiness checker that restricts attention to time-divergent runs; the artifact uses TChecker for this task.
- (C4) *Time constraints in products.* Location invariants are part of the automaton semantics and can block time in a product, thereby removing system behaviors. An exported property automaton is not a passive observer unless its effect on the product is separately justified. Check deadlock or timelock conditions on the system as appropriate.
- (C5) *Numbers.* The integrated tool accepts integer bounds in $[0, 2^{30}]$. The Coq theorems are over real-valued clocks; the extracted implementation's arithmetic and the UPPAAL printer are covered only on the tool's restricted numeric domain.

The generated XML has a `Property` template for the automaton and an `Env` template that can emit events for standalone language exploration. In a system check, the system replaces `Env` and synchronizes its observable events with `Property`; the event `other` denotes any event outside the finite set named in the formula. Accepting locations are named A_n in the output, but those names do not make UPPAAL check Büchi acceptance. For a safety requirement, the reachability query must use an automaton for its violations with a sink entered once the violation has occurred. For a liveness requirement, reachability of such a sink is not the right check and an emptiness procedure for time-divergent Büchi runs is needed.

For a safety property φ , let $\mathcal{A}_{\neg\varphi}$ be a violation automaton whose accepting sink has no invariant and loops on every event. Compose it with a system S that synchronizes the same event alphabet and can extend every queried prefix to a time-divergent execution. Under (C1)–(C3) and the appropriate initialization condition in (C2), the query $E \langle \rangle P.A$ detects a system behavior violating φ when the automaton recognizes exactly those violations. This query-level equivalence is an argument about the chosen system and automaton; it is not proved by the generic Coq optimization or export theorem. The case-study models below report the model checks and diagnostic traces for the four packaged systems.

E. AVAILABILITY

This project contains proof sources, extracted implementation, benchmark tables and raw data, reproduction scripts, and UPPAAL case-study models, together with the LaTeX source and build instructions. The authors' earlier `mtl2tba`

v1.10 tool and proof artifacts document the integrated MTL-to-TBA workflow [3], [4]; this article focuses on its generic TBA-level optimization and export contracts. The invariant, optimization, and export formalization files contain the generic pass results; their theorem names are listed in Table 1. The project repository is at <https://github.com/efares1/verified-tba-optimization-uppaal>.

F. BUILD AND ARTIFACT LAYOUT

The verified functions are defined under `proof/` and extracted into `tool/src/optim.ml`. The project Makefile compiles the Rocq development and extracts the executable functions. Hand-written components include the parser, Spot invocation, LBTT reader, iteration driver, and XML/DOT printers; the script that removes unreachable extracted definitions and the Rocq/OCaml toolchains are also part of the implementation's trusted base. The repository includes the machine record, raw benchmark runs, TSV summaries, table generator, formulas, UPPAAL models and queries, and reproduction instructions in `README.md`.

Table 3 separates the verified transformations from the interfaces that connect them to the surrounding tools.

In the extracted code, structural equality decides formula and clock identity, and Boolean checks implement conditions whose soundness is proved in Coq. Real comparisons are realized by OCaml floating-point operations. Input constants lie in $[0, 2^{30}]$; the inspected operations produce only such constants, their differences, or their negations. These values are exactly representable by the 53-bit significand of a double, and the printed constants fit UPPAAL's 32-bit integer domain. This arithmetic argument is based on inspection of the extracted implementation rather than a theorem about OCaml or the printer. Rational constants can be handled by a common time scaling only when every scaled bound remains an integer in $[0, 2^{30}]$; timing constants in any surrounding system model must be scaled by the same factor.

VI. EVALUATION

A. QUESTIONS AND SETUP

The evaluation addresses three questions: (1) what structural changes do optimization and export make on the current tool's benchmark automata; (2) how does the pipeline scale on conjunctions of bounded responses and nested bounded eventualities; and (3) do the output models support the included UPPAAL and timed-language checks? These are artifact and benchmark questions. The evaluation does not compare downstream checker runtime or memory before and after preprocessing; the measured benefit is verified normalization, output compatibility, and the structural changes reported below. The correctness theorems are universal in the input TBA, but measured inputs come from the current MTL-to-TBA workflow.

The tracked benchmark contains 31 configurations: requirement patterns F1–F14 and F3b; R_n , which conjoins n bounded responses for $1 \leq n \leq 8$; and N_n , a nested bounded-eventuality family for $1 \leq n \leq 8$. The evaluation script

attempts each configuration up to five times with a 300-second limit and stops after the first failed run. Successful configurations have five runs; R7 and R8 each have one timeout attempt. Reported stage and total times are medians of the successful runs. Full structural rows, run ranges, memory, and back-end comparisons appear in Tables 4 and 6; exact formulas are in Table 5. Runs used Ubuntu 26.04 under WSL 2 on an Intel Core i7-8650U at 1.9 GHz with 11.7 GB memory, Rocq 9.0.1, OCaml 5.4.0, and Spot 2.16. CASAAL, an MTL-to-timed-Büchi translator [5], ran natively on Windows on the same machine, so timing comparisons across operating systems are indicative only. UPPAAL case studies used version 5.0.0.

Table 4 gives the complete structural and timing results for all 31 configurations; exact input formulas are listed in Table 5.

B. BENCHMARK INPUTS

The complete list of the 31 input formulas is given in Table 5. The families R_n and N_n make the scaling dimensions explicit: R_n is a conjunction of n independent bounded responses, while N_n nests n bounded eventualities. The table records the exact formulas passed through the integrated tool and identifies the two generated-automaton examples discussed in Section IV. These inputs characterize the measured benchmark; they do not restrict the generic preservation theorems.

C. STRUCTURAL EFFECT OF THE PASSES

The post-processing reduces transitions by 96% in R6 (15,625 to 640), 82% in R4 (625 to 112), and 53% in F13 (17 to 8); R5 falls from 3,125 to 272. These compare the upstream automaton with the output of the post-processing chain, which includes reset completion in this tool. They illustrate the observed benefit on generated inputs, not a reduction bound for arbitrary TBAs.

Export can reverse part of that reduction because it expands disjunctive guards into explicit conjunctive edges and adds entry conditions to invariant copies. R4 grows from 112 optimized edges to 217 exported edges, with 192 difference-constraint occurrences. R6 grows from 640 to 1,837 edges, with 3,360 such occurrences. On N4, 25 explicit edges become 11 symbolic transitions. The number of locations does not change in these examples. Thus export trades compact disjunctions for an explicit conjunctive representation suitable for UPPAAL-oriented output.

The pipeline removes one of two synchronously reset clocks in F11. Other clock-count changes in the full table can arise upstream: for example, the default formula rewrite changes F8 before the generic optimizer sees the automaton. We do not attribute such changes to generic optimization unless a pass is responsible.

The size columns record distinct stages because their counts answer different questions. Spot's transition count is measured after splitting its Boolean labels into cubes. The optimized count follows reset completion and all verified

TABLE 3. Runtime stages and their proof boundary. The generic theorems begin with a TBA in the Coq representation; the Spot-based formula front end is one client of the verified optimizer and exporter.

Stage	Implementation	Guarantee or boundary
TBA input and adapter	Coq TBA value or a parser/converter supplied by the caller	Generic pass results start at this representation; an external adapter is outside the theorem unless separately verified.
Spot-based front end	Spot call and hand-written LBTT reader in <code>mtl2tba</code>	Used by the integrated formula workflow; outside the generic TBA theorem.
Optimization rounds	Extracted from <code>MTL_to_TBA_Optimizations.v</code>	<code>optimize_accepts</code> , for every TBA and every round count.
Iteration driver	Hand-written OCaml structural comparison; stops at equality or 50 rounds	The proof does not assume convergence or a particular stopping point.
UPPAAL export	Extracted from <code>MTL_to_TBA_Export.v</code>	<code>export_accepts</code> and conjunctive guard/invariant results.
Symbolic grouping	Extracted from <code>MTL_to_TBA_Symbolic.v</code>	<code>symbolic_ext_accepts</code> .
XML, DOT, and PDF output	Hand-written printers and Graphviz	Outside the Coq language-preservation results.
Numeric realization	OCaml floating point with integer bounds in $[0, 2^{30}]$	Exactness follows from inspection of the extracted operations, not a mechanized theorem.

optimization rounds; the exported count additionally expands disjunctions to meet the conjunctive representation. The symbolic count groups those explicit edges by source, target, and reset set while retaining their Boolean labels. Consequently, R6 falls from 15,625 Spot transitions to 640 after optimization, grows to 1,837 explicit exported transitions, and then groups to 448 symbolic transitions. The last count matches CASAAL's 448 transitions on the common domain, while the explicit export also contains 3,360 occurrences of clock-difference constraints. These figures expose the tradeoff between simplifying the input and making disjunctions explicit for export; they are not contradictory measurements of one representation.

D. COMPARISON DOMAIN AND RELATIVE SIZES

CASAAL accepts sets of propositions at each position, while the benchmark tool uses one event per position. For cross-tool comparisons, each CASAAL formula is conjoined with mutual-exclusion constraints for every pair of distinct propositions; positions where none holds represent the event *other*. On that common domain, F2 and F3b reduce to $\Box \neg p$ and $\Box \neg e$. F3 and F13 use hatted operators without a CASAAL counterpart, so their CASAAL automata are reported only as nearby encodings and are not equivalent-language comparisons.

Among the 27 configurations that both tools translate and whose formulas are comparable, the symbolic automata have equal location and transition counts on 24: the nested family N1–N8, R1–R6, and F1, F2, F3b, F4, and F7–F12. The verified pipeline is smaller on F14 (6 locations and 14 symbolic transitions, versus 8 and 18) and larger on F5 and F6 (7 locations and 25 transitions, versus 4 and 15). These two lower-bound response cases retain the fairness branch of the hatted Until encoding upstream of the generic passes. The exported conjunctive edge counts can be much larger than the symbolic counts because each disjunctive guard is split into explicit transitions; the symbolic representation groups them again for comparison. These size observations describe this

benchmark, not a bound on either construction.

E. SCALING AND BACK-END CHOICES

For the bounded-response family, R6 completes in 36.75 seconds, of which optimization takes 16.57 seconds; R7 and R8 exceed the 300-second limit. The nested family reaches N8 in 103.02 seconds and 961 MB, with Spot taking 101.42 seconds. Optimization takes 0.24 seconds on N8; most time is in the upstream Spot construction. Spot's transition-cube representation grows rapidly in Rn , and every later pass processes it. Sharing Boolean subformulas or retaining Boolean labels earlier are possible changes, but their effect was not measured.

Table 6 gives stage times, run ranges, peak memory, and the exported transition counts under the default and determinism-preference Spot options. Table 7 isolates the effect of the weak-until rewrite. The determinism-preference option grows N4 from 6 locations and 25 optimized transitions to 34 locations and 1,035 transitions and times out on larger nested instances. These are configuration measurements, not proof conditions or asymptotic bounds. For F9, F10, and F14, the same determinism preference changes the output from 4 locations and 9 transitions to 6 and 25, from 5 and 13 to 7 and 31, and from 6 and 19 to 11 and 60, respectively; N6 exceeds the time limit. With weak Until disabled, Spot instead receives strong Until in the bounded clauses. On R4 this grows the returned automaton from 16 to 48 states and raises the whole-chain median from 0.41 to 2.14 seconds; on R5 it grows from 32 to 112 states and from 2.90 to 37.48 seconds, while R6 exceeds the limit. This is an observed Spot 2.16 degeneralization effect on these formulas. The weak-until clause does not weaken the timed property: time divergence discharges the bounded upper-window obligation. Neither the determinism nor the weak-until option is a premise of the Coq optimization or export proofs.

Spot dominates the measured N8 runtime: the whole chain takes 103.02 seconds, of which 101.42 seconds are in Spot, and the run uses 961 MB. The printed clocked-LTL tree

TABLE 4. Evaluation of `mt12tba` and comparison with CASAAL on the common semantic domain (one event per position). $|X|$: number of distinct primitive (hatted) timed subformulas after unfolding of the ordinary operators and the default rewritings. Spot: Büchi automaton returned by Spot (states/transitions, one transition per cube). Opt.: after reset completion and the verified optimizations (locations/transitions/clocks). Export: automaton for UPPAAL (locations, transitions, clocks, locations with an invariant, and occurrences of difference constraints in the guards). Sym.: transitions of the symbolic automaton, with Boolean labels. Time: wall-clock time, median of five runs of the whole chain including Spot, in seconds. st, tr, loc, clk: states, transitions, locations, clocks. CASAAL: states/transitions/clocks of the automaton of CASAAL for the formula conjoined with the mutual exclusion of its propositions; transitions carry Boolean formulas. †: not equivalent (CASAAL has no hatted operator); ‡: degenerate formula, equivalent to $\Box \neg p$ or $\Box \neg e$ with one event per position. CASAAL time: wall-clock time, median of five runs, in seconds, including the start of the process. TO: the chain exceeds the limit of 300 s (wall-clock, per run). Lang.: comparison of the languages with those of CASAAL by TChecker (Section VI, paragraph “Comparison of languages with CASAAL”): exp, inclusion of the languages of CASAAL’s automata into ours established on the exported automaton; cmp, established only on the automaton `compile(f)` before post-processing; ?, no conclusion within the limits; –, formulas not equivalent. Equal sizes do not by themselves imply equal languages. Formulas: Table 5 of the supplementary material.

Id	$ X $	Spot		Opt.			Export					Sym.	Time	CASAAL		Lang.
		st	tr	loc	tr	clk	loc	tr	clk	inv	diff			st/tr/clk	Time	
F1	1	2	5	2	5	1	2	5	1	1	0	4	0.02	2/4/1	0.04	exp
F2 [†]	1	2	5	1	1	0	1	1	0	0	0	1	0.02	1/1/0	0.04	exp
F3	1	2	5	2	5	1	2	5	1	0	0	5	0.02	3/6/1 [†]	0.03	–
F3b [†]	1	1	1	1	1	0	1	1	0	0	0	1	0.02	1/1/0	0.03	exp
F4	1	2	5	2	5	1	2	5	1	1	0	4	0.02	2/4/1	0.05	exp
F5	1	7	38	7	25	1	7	25	1	0	0	25	0.03	4/15/1	0.03	exp
F6	1	7	38	7	25	1	7	25	1	0	0	25	0.03	4/15/1	0.03	exp
F7	1	2	4	2	4	1	2	4	1	1	0	4	0.02	2/4/1	0.03	exp
F8	0	2	4	2	4	0	2	4	0	0	0	4	0.02	2/4/1	0.03	exp
F9	2	4	9	4	9	2	4	9	2	2	0	9	0.02	4/9/2	0.03	exp
F10	3	5	13	5	13	3	5	13	3	2	0	12	0.03	5/12/3	0.03	exp
F11	2	5	11	4	7	1	4	7	1	2	0	7	0.02	4/7/2	0.03	exp
F12	2	4	17	3	9	2	3	9	2	2	0	9	0.02	3/9/2	0.03	exp
F13	2	5	17	3	8	2	3	8	2	2	0	8	0.02	4/7/2 [†]	0.03	–
F14	2	6	26	6	17	2	6	19	2	4	2	14	0.03	8/18/2	0.03	cmp
R1	1	2	5	2	5	1	2	5	1	1	0	4	0.02	2/4/1	0.05	exp
R2	2	4	25	4	16	2	4	19	2	3	2	12	0.02	4/12/2	0.04	cmp
R3	3	8	125	8	44	3	8	67	3	7	30	32	0.04	8/32/3	0.05	cmp
R4	4	16	625	16	112	4	16	217	4	15	192	80	0.25	16/80/4	0.05	?
R5	5	32	3125	32	272	5	32	651	5	31	880	192	2.88	32/192/5	0.14	?
R6	6	64	15625	64	640	6	64	1837	6	63	3360	448	40.87	64/448/6	0.36	?
R7	7							TO						128/1024/7	1.55	?
R8	8							TO						256/2304/8	2.91	?
N1	1	3	5	3	5	1	3	5	1	1	0	5	0.02	3/5/1	0.03	exp
N2	2	4	11	4	9	2	4	9	2	2	0	7	0.02	4/7/2	0.03	exp
N3	3	5	21	5	16	3	5	16	3	3	0	9	0.03	5/9/3	0.03	exp
N4	4	6	36	6	25	4	6	25	4	4	0	11	0.05	6/11/4	0.03	exp
N5	5	7	57	7	36	5	7	36	5	5	0	13	0.41	7/13/5	0.03	?
N6	6	8	85	8	49	6	8	49	6	6	0	15	1.38	8/15/6	0.04	?
N7	7	9	121	9	64	7	9	64	7	7	0	17	9.72	9/17/7	0.04	?
N8	8	10	166	10	81	8	10	81	8	8	0	19	111.98	10/19/8	0.04	?

grows from 385 characters for N2 to 33,145 for N8 (24 distinct atoms); identical subformulas are expanded in the text interface. A shared-subformula representation may reduce this input string, but its effect on Spot’s runtime has not been measured. CASAAL translates N8 in under 0.1 seconds on the recorded machine. These timings compare different implementations and operating systems and should be read as artifact measurements, not as a general complexity comparison. The timing profile differs on R6: Spot takes 6.36 seconds, optimization 16.57 seconds, and export 10.31 seconds. Here the large input transition relation makes the verified passes themselves a substantial cost, even though the resulting optimized relation is much smaller. In contrast, on N8 the optimizer and exporter together take less than one second, and the upstream translation dominates. Thus neither

a single pipeline stage nor a single scaling pattern explains the whole benchmark. The stage times help locate the current engineering bottlenecks but do not predict performance on automata from other producers.

F. ACCEPTANCE REPRESENTATION AT THE SPOT BOUNDARY

The integrated tool requests state-based Büchi acceptance from Spot. The acceptance-representation experiment below isolates another input-size choice on the bounded-response family. Here, “weak” uses the default weak-until clauses in the bounded translation; “strong” uses ordinary Until instead. Each entry reports Spot states and acceptance sets, from the raw results in `evaluation/acceptance_results.tsv`.

TABLE 5. Benchmark formulas, in the input syntax of `mtl2tba` ($\neg$, $\wedge$, $\vee$, $\rightarrow$, $\rightarrow$: hatted operators; $[]$, $\langle\rangle$, $\square$, $\diamond$; $U[\leq 2]$: Until with bound ≤ 2 ; $!$, $\&$, $|$, $\rightarrow$: negation, conjunction, disjunction, implication). F3 and F3b use the non-strict bound ≤ 2 . CASAAL receives the same formula, with $\wedge$ and $\vee$ for $\wedge$ and $\vee$, conjoined with $[]$ ($!(a \wedge b)$) for every pair of distinct propositions; for F3 and F13 it receives the closest encodings with a leading $\diamond$, which are not equivalent. R1 is F1 with renamed propositions; it is the first instance of the family R_n .

Id	Property	Formula
F1	bounded response	$[] (p \rightarrow \langle\rangle [\leq 5] q)$
F2	bounded invariance	$[] (p \rightarrow [] [\leq 3] q)$
F3	sporadicity (hatted)	$[] (e \rightarrow \hat{[]} [\leq 2] !e)$
F3b	sporadicity, no hat	$[] (e \rightarrow [] [\leq 2] !e)$
F4	time-bounded Until	$[] (p \rightarrow (p U[\leq 4] q))$
F5	minimum-delay response	$[] (p \rightarrow \langle\rangle [\geq 2] q)$
F6	strict minimum-delay response	$[] (p \rightarrow \langle\rangle [\geq 2] q)$
F7	bounded recurrence	$[] \langle\rangle [\leq 5] p$
F8	recurrence with minimum delay	$[] \langle\rangle [\geq 2] p$
F9	Example S1	$\langle\rangle [\leq 5] (p U[\leq 2] q)$
F10	Example S2	$\langle\rangle [\leq 5] (([] [\leq 3] (p q)) U[\leq 2] q)$
F11	Running example	$(\langle\rangle [\leq 5] p) \& ([] [\leq 2] !p)$
F12	response within a window	$[] (p \rightarrow (\langle\rangle [\leq 5] q \& [] [\leq 2] !q))$
F13	periodicity (hatted)	$e \& [] (e \rightarrow (\hat{[]} [\leq 2] !e \& \langle\rangle [\leq 2] e))$
F14	closeness	$[] (e1 \rightarrow (\langle\rangle [\leq 2] (e2 \& \langle\rangle [\leq 3] e3) [] (!e2 \& !e3)))$
R1	1 bounded response	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1)$
R2	2 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2)$
R3	3 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle\rangle [\leq 5] b3)$
R4	4 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle\rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle\rangle [\leq 5] b4)$
R5	5 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle\rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle\rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle\rangle [\leq 5] b5)$
R6	6 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle\rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle\rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle\rangle [\leq 5] b5) \& [] (a6 \rightarrow \langle\rangle [\leq 5] b6)$
R7	7 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle\rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle\rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle\rangle [\leq 5] b5) \& [] (a6 \rightarrow \langle\rangle [\leq 5] b6) \& [] (a7 \rightarrow \langle\rangle [\leq 5] b7)$
R8	8 bounded responses	$[] (a1 \rightarrow \langle\rangle [\leq 5] b1) \& [] (a2 \rightarrow \langle\rangle [\leq 5] b2) \& [] (a3 \rightarrow \langle\rangle [\leq 5] b3) \& [] (a4 \rightarrow \langle\rangle [\leq 5] b4) \& [] (a5 \rightarrow \langle\rangle [\leq 5] b5) \& [] (a6 \rightarrow \langle\rangle [\leq 5] b6) \& [] (a7 \rightarrow \langle\rangle [\leq 5] b7) \& [] (a8 \rightarrow \langle\rangle [\leq 5] b8)$
N1	nesting depth 1	$\langle\rangle [\leq 5] p1$
N2	nesting depth 2	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] p2)$
N3	nesting depth 3	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] (p2 \& \langle\rangle [\leq 5] p3))$
N4	nesting depth 4	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] (p2 \& \langle\rangle [\leq 5] (p3 \& \langle\rangle [\leq 5] p4)))$
N5	nesting depth 5	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] (p2 \& \langle\rangle [\leq 5] (p3 \& \langle\rangle [\leq 5] (p4 \& \langle\rangle [\leq 5] p5))))$
N6	nesting depth 6	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] (p2 \& \langle\rangle [\leq 5] (p3 \& \langle\rangle [\leq 5] (p4 \& \langle\rangle [\leq 5] (p5 \& \langle\rangle [\leq 5] p6))))$
N7	nesting depth 7	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] (p2 \& \langle\rangle [\leq 5] (p3 \& \langle\rangle [\leq 5] (p4 \& \langle\rangle [\leq 5] (p5 \& \langle\rangle [\leq 5] (p6 \& \langle\rangle [\leq 5] p7))))))$
N8	nesting depth 8	$\langle\rangle [\leq 5] (p1 \& \langle\rangle [\leq 5] (p2 \& \langle\rangle [\leq 5] (p3 \& \langle\rangle [\leq 5] (p4 \& \langle\rangle [\leq 5] (p5 \& \langle\rangle [\leq 5] (p6 \& \langle\rangle [\leq 5] (p7 \& \langle\rangle [\leq 5] p8))))))$

With strong Until, Spot's generalized transition-based output has 2^n states and n acceptance sets for R_n in these completed cases. The single-set transition-based form uses fewer states than the state-based form for R4–R6, but still grows from 40 states for R4 to 224 for R6. With the weak-until clauses, the returned state counts are 2^n and the generalized output has no acceptance sets. These runs show that acceptance encoding and the bounded-clause rewrite both affect the automaton received by the optimizer. The integrated pipeline uses state-based output, so the other rows are diagnostic

ablations rather than alternative verified tool configurations; the generic preservation theorem is stated after conversion to the Coq TBA representation.

The last column also shows why state counts alone understate the optimizer's work. The corresponding Spot outputs have 256, 1,024, and 4,096 labeled edges; the integrated reader expands their Boolean labels into 625, 3,125, and 15,625 transition cubes. Each optimization round processes those cubes before symbolic grouping is restored at export. This label expansion is a measured source of work in the response family. Retaining Boolean labels through the veri-

TABLE 6. Scalability on the families R_n and N_n : median time of the Spot, optimization, and export stages and of the whole chain, minimum and maximum of the five runs, peak memory, and exported transitions with Spot options `-B -small` (default) and `-B -D`. Times in seconds, memory in MB. The total also includes the stages not listed: reading of the output of Spot, reset completion, check `init_free`, symbolic transitions, and printing.

Id	Spot	Opt.	Exp.	Total	[min, max]	Mem.	tr	tr -D
R1	0.01	0.00	0.00	0.02	[0.02, 0.04]	17	5	5
R2	0.01	0.00	0.00	0.02	[0.02, 0.05]	17	19	19
R3	0.01	0.00	0.01	0.04	[0.04, 0.06]	18	67	67
R4	0.05	0.04	0.11	0.25	[0.24, 0.36]	19	217	217
R5	0.46	0.67	1.22	2.88	[2.75, 3.43]	23	651	651
R6	6.83	18.21	11.98	40.87	[39.29, 45.75]	106	1837	1837
R7	timeout (limit 300 s)							
R8	timeout (limit 300 s)							
N1	0.01	0.00	0.00	0.02	[0.02, 0.04]	17	5	5
N2	0.01	0.00	0.00	0.02	[0.02, 0.04]	17	9	26
N3	0.01	0.00	0.00	0.03	[0.03, 0.04]	18	16	173
N4	0.03	0.00	0.00	0.05	[0.05, 0.07]	19	25	1035
N5	0.35	0.01	0.02	0.41	[0.36, 0.47]	22	36	5461
N6	1.24	0.02	0.06	1.38	[1.32, 2.83]	35	49	timeout
N7	9.22	0.09	0.22	9.72	[8.91, 12.03]	140	64	timeout
N8	109.70	0.29	0.73	111.98	[107.67, 137.38]	961	81	timeout

TABLE 7. Effect of the weak until (default; disabled by the option `-noweak`), on the configurations whose sizes change. Spot: states of the automaton returned by Spot; Sym.: symbolic transitions; Time: median of five runs, in seconds; TO: more than 300 s; CASAAL: states/transitions on the common domain, [†] as in Table 4. The clocks do not change.

Id	-noweak			default			CASAAL
	Spot	Sym.	Time	Spot	Sym.	Time	
F1	3	6	0.02	2	4	0.02	2/4
F4	3	6	0.03	2	4	0.02	2/4
F7	3	6	0.02	2	4	0.02	2/4
F12	5	13	0.05	4	9	0.02	3/9
F13	6	9	0.03	5	8	0.02	4/7 [†]
F14	8	20	0.04	6	14	0.03	8/18
R1	3	6	0.02	2	4	0.02	2/4
R2	8	21	0.04	4	12	0.02	4/12
R3	20	68	0.23	8	32	0.04	8/32
R4	48	205	2.14	16	80	0.25	16/80
R5	112	582	37.48	32	192	2.88	32/192
R6	TO			64	448	40.87	64/448

fied passes could avoid some repeated processing, but such a change would require new preservation results and has not been evaluated.

G. LANGUAGE CHECKS AND COMPARISON LIMITS

H. TRANSLATION VALIDATION AND LANGUAGE COMPARISONS

The repository includes scripts and raw TSV results for the following independent checks. For CASAAL comparisons, TChecker 0.8 checks products of our automata O, O' for a formula and its negation with CASAAL automata C, C' . The checks seek emptiness of $O \times C'$, $C \times O'$, and $O \times O'$, with additional processes enforcing one event per position, strictly increasing timestamps, and divergent time. CASAAL transition-based acceptance is encoded with committed intermediate locations. This conversion script is unverified. Since

timed Büchi automata cannot in general be complemented, empty products establish inclusions under the stated assumptions; language equality also depends on the correctness of CASAAL's complement automaton and is not claimed here.

- **Spot-output cross-check.** An independent translator (ltl2ba 1.3) and Spot's `autfilt` check the returned automaton against the clocked-LTL input. Both inclusions conclude on 25 configurations; on R6–R8 and N6–N8, ltl2ba does not finish within 300 seconds. This is benchmark translation validation, not a verified replacement for the LTL-to-Büchi translator.
- **Interface check.** With `-check`, Spot parses the formula from the ordinary printer and a second prefix printer, and `autfilt -equivalent-to` compares the automaton read by the tool with Spot's raw output. Checks pass on all 31 configurations. A common error in both printers or Spot's parser is not ruled out.
- **Random formulas.** The repository records 100 generated formulas of depth at most three over three propositions. With a 120-second bound per product check, comparison products are empty for 95 formulas; for four others, checks that finish report emptiness. One remaining formula produces a nonempty product related to CASAAL's printed acceptance marking for an unfulfilled eventuality, as documented in `evaluation/README.md`.
- **Implementation checks.** Eighteen finite-word simulator checks exercise strict and non-strict boundaries and overlapping activations. They pass and validate selected output behaviors; they do not replace the mechanized proof.

The remaining random-formula case is $\Diamond(q \wedge r) \Rightarrow \neg p$. On the common one-event domain, its negation $\Diamond(q \wedge r) \wedge p$ is unsatisfiable because q and r cannot both hold at one position; our automaton therefore has no accepting location. The CASAAL output read by the conversion script has no acceptance mark for this unfulfilled eventuality, and the script interprets an unmarked automaton as accepting every infinite run. The resulting nonempty product is thus a format/acceptance ambiguity in this comparison. The available output does not establish whether CASAAL retains an empty acceptance set internally, so this case is reported as inconclusive rather than as a disagreement between the tools.

I. TIMED-LANGUAGE COMPARISON WITH CASAAL

The table's language column records inclusions checked with TChecker 0.8, using its `couvsec` timed Büchi emptiness algorithm. Write O and O' for our automata for a formula φ and its negation, and C and C' for the corresponding CASAAL automata. The checks ask whether $O \times C'$, $C \times O'$, and $O \times O'$ are empty; as a sanity check, $O \times C$ must be nonempty. Two auxiliary processes restrict products to one event per position, strictly increasing timestamps, and divergent time. Because CASAAL marks acceptance on transitions, its output is converted to state-based acceptance with committed intermediate locations. The format conversion is a repository script and is not verified.

TABLE 8. Spot acceptance representations for R_n . The state-based ($-B$), single-set transition-based ($-b$), and transition-based generalized ($- - tgba$) modes are shown for weak- and strong-until clauses. Entries are states/acceptance sets. The final column counts transition cubes produced by the integrated reader from the weak-until state-based output.

	Weak $-B$	Weak $- - tgba$	Strong $-B$	Strong $-b$	Strong $- - tgba$	Weak $-B$ reader cubes
R_4	16/1	16/0	48/1	40/1	16/4	625
R_5	32/1	32/0	112/1	96/1	32/5	3,125
R_6	64/1	64/0	256/1	224/1	64/6	15,625

Assuming the verified theorem and a correct Spot automaton, O and O' are complementary. Emptiness of $O \times C'$ then establishes $C' \subseteq O'$, and emptiness of $C \times O'$ establishes $C \subseteq O$. These are inclusions from CASAAL into the verified tool's languages. They do not establish equality: timed Büchi automata cannot in general be complemented, and the correctness of CASAAL's automata is not proved here. The $O \times O'$ check serves as a consistency check on the converted formats and the product setup.

Of the 29 configurations whose formulas are comparable, all checks conclude as expected on the exported automata for 17. On F14, R2, and R3, the exported automata contain clock-difference constraints and TChecker does not conclude within 600 seconds; the checks conclude on the automata after reset completion but before optimization. For R4–R6, TChecker remains inconclusive even on those pre-optimization automata. It also does not conclude for N5 and N6, while the integrated tool exceeds its own 300-second limit on R7, R8, and the negations of N7 and N8. Thus 3 configurations have an inclusion established only before post-processing and 9 have no conclusion within the limits; no completed check reports a disagreement.

The pre-optimization comparisons are observations rather than consequences of the Coq pass theorem. The products initialize clocks to zero, whereas the generic preservation contract existentially quantifies over initial clock values, and the product checker and format conversion are outside the proof. We report these results as translation validation on the tracked benchmark, not as a second proof of the optimization or export pipeline.

J. UPPAAL CASE STUDIES

1) Property automata

For sporadicity, the violation formula $\Diamond(e \wedge \hat{\Diamond}_{<2} e)$ is compiled into locations $L0$ (initial), $A2$ (accepting, invariant $x_0 \leq 2$), and accepting sink $A4$, with one clock:

- $L0 \rightarrow L0$: every event / x_0 ; $L0 \rightarrow A2$: e / x_0 ;
- $A2 \rightarrow A2$: $x_0 < 2$, *other*; $A2 \rightarrow A4$: $x_0 < 2$, e ;
- $A4 \rightarrow A4$: every event.

The automaton guesses one e and reaches $A4$ when a second follows less than two time units later. The bounded accepting location cannot be occupied forever on a time-divergent word, so acceptance entails a visit to the violation sink.

For bounded response, the violation formula is $\Diamond(p \wedge \hat{\Box}_{\leq 5} \neg q)$, with locations $L0$ (initial), $A2$ (accepting), and $A4$ (accepting sink):

TABLE 9. Case study with UPPAAL 5.0.0. Reachability is $E<> P.A4$; no deadlock is $A[] \text{ not deadlock}$ on the system alone.

System	Reachability	Diagnostic trace	No deadlock
Emitter, $k = 2$	no	—	yes
Emitter, $k = 1$	yes	$(e, 1)(e, 2)$	yes
Controller, $d = 4$	no	—	yes
Controller, $d = 6$	yes	$(p, 0.5)(q, 6)$	yes

- $L0 \rightarrow L0$: every event; $L0 \rightarrow A2$: p / x_0 ;
- $A2 \rightarrow A2$: p or *other*; $A2 \rightarrow A4$: $x_0 > 5$, every event;
- $A4 \rightarrow A4$: every event.

A run that remains in $A2$ forever corresponds to a request that is never answered. On a word with divergent time and infinitely many events, an event eventually enables the transition to $A4$.

2) System models and results

Every system has a clock z , reset on every emission and tested by $z > 0$ so event times strictly increase, and a clock y for its timing parameter.

- *Sporadic emitter*: A single location S has $S \rightarrow S$: $y \geq k \wedge z > 0$, $e / y, z$, and $S \rightarrow S$: $z > 0$, *other* / z . It satisfies the requirement for $k = 2$ and violates it for $k = 1$.
- *Controller*: Locations are *Idle* (initial) and *Busy*, with invariant $y \leq d$. The transition *Idle* $\rightarrow$ *Busy* emits p when $z > 0$ and resets y, z ; *Idle* can emit *other* when $z > 0$ and reset z . In *Busy*, *other* is emitted under $z > 0 \wedge y < d$, resetting z , and q is emitted under $z > 0 \wedge y \geq 1$, resetting z and returning to *Idle*. It satisfies bounded response for $d = 4$ and violates it for $d = 6$.

The query is $E<> P.A4$. Table 9 records UPPAAL 5.0.0 results. The system-alone deadlock check composes each system with a receiver of every channel, without the property automaton. A controller variant without guard $y < d$ can emit *other* at $y = d$, after which the $z > 0$ test prevents another event while the invariant prevents time advancing. UPPAAL's system-alone $A[] \text{ not deadlock}$ query detects this timelock; packaged models use the corrected guard.

The diagnostic traces explain the two positive reachability results. With $k = 1$, the emitter can produce e at times 1 and 2, so the second event is strictly less than two units after the first and reaches the violation sink. With $k = 2$, every pair of consecutive emissions is separated by at least two units, so the sink is unreachable. For the controller with $d = 6$, the trace $(p, 0.5)(q, 6)$ leaves 5.5 time units between request

and response, exceeding the five-unit bound. When $d = 4$, the invariant and response transition force an answer no later than four units after the request. The “no deadlock” column reports a separate system-only query; it rules out reachable deadlock states in these models but does not itself prove time divergence. The models also admit continuations with positive, unbounded elapsed time, as required for interpreting a finite violation prefix as an infinite timed word.

K. A LIVENESS CHECK WITH TCHECKER

For the unbounded-response property $\Box(p \Rightarrow \Diamond q)$, a violation automaton for $\Diamond(p \wedge \Box\neg q)$ has two locations and no clocks. Composed with a system that answers every request within four time units, the product has no accepting time-divergent run. If the system can remain busy forever after a request, the product contains an accepting cycle, representing a liveness violation. The artifact checks these products with TChecker’s timed Büchi emptiness procedure. This example complements the UPPAAL reachability checks: it uses the acceptance condition that ordinary UPPAAL reachability does not express.

L. IMPLICATIONS OF THE RESULTS

The universal preservation results and the measured performance answer different questions. The optimizer and exporter preserve every TBA presented to their Coq interface, and the optimization proof holds for any finite number of rounds. This removes a dependence on the integrated driver’s choice to stop at structural equality or at 50 rounds. It does not give a bound on the size or runtime of the output, and it does not extend the proof across an unverified format adapter.

The evaluation identifies two distinct costs in the current workflow. For the bounded-response family, Spot’s labeled edges expand into many transition cubes before optimization; the passes then spend substantial time reducing that representation. For nested bounded eventualities, the upstream Spot construction dominates and the optimization/export stages remain short. This distinction matters when choosing where to improve the implementation: keeping Boolean labels through verified passes may help the response family, whereas sharing repeated subformulas before printing the Spot input may help the nested family. Neither change has been implemented or measured here.

The explicit and symbolic outputs serve different consumers. Explicit export meets the conjunctive guard and invariant shape used by UPPAAL-oriented XML, but splitting disjunctions can increase edge counts and introduce many clock-difference constraints. Symbolic grouping recovers compact Boolean labels for drawings and comparisons; it does not remove the explicit constraints from the exported model. The measured R6 counts therefore motivate reporting both forms instead of treating transition count as a single pipeline metric.

Finally, a third-party TBA can use the generic theorem only after its labels, resets, acceptance marking, and starting-clock convention have been mapped to the Coq semantics.

The repository contains no verified reader for such an external format, and the performance study does not yet test independently produced automata. Building verified adapters and evaluating on independent TBA corpora are the next steps needed to turn the generic result into a measured reusable tool for other automaton producers.

M. THREATS TO VALIDITY

The benchmark is fixed, hand-selected, and generated through one MTL-based toolchain; it contains no industrial specification and is not a broad arbitrary-TBA suite. R7/R8 time out, and output sizes are sensitive to Spot version and options. The CASAAL language comparison is limited by operator coverage, common-domain encoding, product-checker limits, and an unverified conversion script. The checked-in results do not identify the exact CASAAL executable or checksum; `evaluation/run_casaal.py` now writes a SHA-256 manifest for the supplied executable and DLLs on future runs, but the version still needs to be recorded. Although `evaluation/machine.txt` records the versions used for the article, `evaluation/setup_wsl.sh` installs from mutable Ubuntu and Spot repositories and does not pin those versions. The tools ran under different operating systems. The case study consists of four small parameterized models. These limitations concern empirical generality; they do not narrow the universal quantification of the Coq preservation theorems.

VII. RELATED WORK

The work relates to semantics-preserving simplification of timed automata, disjunctive invariants in UPPAAL, and mechanically verified automata software.

For invariant synthesis and pruning, Badban, Leue, and Smaus present the CIPM algorithm, which derives location invariants from constraints on incoming transitions and removes idle transitions [6]. They also mention location splitting for disjunctive invariants as a possible direction. We instead synthesize upper bounds from outgoing guards and propagate successor requirements backward; our proof covers timed-word language preservation for arbitrary TBAs and every finite number of optimization rounds. For clock reduction, Daws and Yovine detect active clocks and clocks that remain equal [7]. Our passes remove reset commands that cannot affect future guards and merge clocks only under synchronous-reset equivalence. The NWPT 2003 contribution by Bodeveix and Filali is listed as a work on disjunctive invariants in UPPAAL [2]; the specific result established here is a split with maximal-delay entry conditions and a proof of timed-word language preservation. Behrmann, David, and Larsen’s UPPAAL tutorial provides background on the target tool [1].

The split addresses a specific semantic issue in adapting disjunctive invariants to a target that accepts only conjunctions. Choosing any true disjunct on entry is insufficient: that choice can impose a shorter residence time than another disjunct and remove a timed word. The entry condition in Section III-G compares the delays admitted by the candidate

and competing invariants, and the proof reconstructs a split run by selecting a copy that admits a maximal delay. The split complements our invariant-synthesis pass, which derives upper bounds from outgoing guards; the split theorem handles the disjunctions that arise when guards are merged for export.

Tripakis studies emptiness checking for timed Büchi automata on simulation graphs [8]. UPPAAL and TChecker provide analysis engines, while this work addresses a verified normalization and export layer before analysis. TChecker is used as an external emptiness checker for selected comparisons and a liveness task, not as part of the Coq proof [9].

Mechanized automata verification includes verified LTL-to-Büchi translation and model checking in Isabelle/HOL, including CAVA [10], [11], and verified timed-automata model checking in Munta [12], [13]. Those tools address upstream translation or downstream checking rather than the same transformation task. The present proofs are stated at a TBA interface; using the output with another checker still requires a format and semantics bridge. The extraction mechanism for the OCaml implementation is described by Letouzey [14]. This separation also clarifies the role of the experiments. TChecker and UPPAAL are downstream consumers used to exercise selected exported models; their verdicts do not establish the Coq preservation results. Conversely, the Coq results do not prove the external parser, format conversion, or checker implementation correct. The verified component is a reusable transformation layer between an automaton producer and a checker, with explicit adapters at both ends. Extending those adapters with verified format correspondence is a natural way to compose this work with independently verified translators or model checkers.

VIII. CONCLUSION

We presented a Coq-verified optimization and export pipeline for timed Büchi automata. The transformations synthesize and propagate location invariants, simplify timing constraints, remove infeasible edges and dead resets, merge synchronously reset clocks and equivalent automaton structure, and eliminate disjunctive invariants to produce conjunctive guards and invariants. The optimization theorem is generic in its TBA input and holds for every number of rounds; export preserves the same language and has verified output-shape properties.

The extracted implementation is used by `mtl2tba`. Tracked measurements show substantial transition reductions on several generated examples, while also exposing the cost of expanding disjunctions for explicit UPPAAL output and the scaling limits of the upstream automata. Four UPPAAL case-study models produce the expected safety-query verdicts. These measurements characterize the existing benchmark and tool chain; they do not establish performance over a broad collection of arbitrary TBAs.

Future work includes evaluating the generic passes on independent TBA corpora, improving symbolic handling of Boolean transition labels before Spot expansion, extending clock merging beyond synchronous reset, and verifying addi-

tional input/output adapters so third-party formats can use the pipeline with smaller trusted interfaces.

REFERENCES

- [1] G. Behrmann, A. David, and K. G. Larsen, “A tutorial on UPPAAL,” in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3185, pp. 200–236.
- [2] J.-P. Bodeveix and M. Filali, “About disjunctive invariants in UPPAAL (NWPT, Turku Finlande, 29/10/03-31/10/03),” in *NWPT, Turku Finlande, 29/10/03-31/10/03*, octobre 2003. [Online]. Available: <http://www.abo.fi/~Kaisa.Sere/15NWPT.html>
- [3] E. Fares and J.-P. Bodeveix, “mtl2tba: A verified translator from $MTL_{0,\infty}$ to timed Büchi automata,” GitHub repository, release v1.10, 2026. [Online]. Available: <https://github.com/efares1/mtl2tba/releases/tag/v1.10>
- [4] —, “mtl2tba: Coq development of the correctness proof of the $MTL_{0,\infty}$ -to-TBA translation,” GitHub repository, release v1.10, directory proof, 2026. [Online]. Available: <https://github.com/efares1/mtl2tba/tree/v1.10/proof>
- [5] G. Li, “CASAAL: A tool for translating $MTL_{0,\infty}$ into timed Büchi automata,” Online, 2017, available at the CASAAL tool page. [Online]. Available: <https://lcs.ios.ac.cn/~ligy/tools/CASAAL/>
- [6] B. Badban, S. Leue, and J.-G. Smaus, “Automated invariant generation for the verification of real-time systems,” in *WING 2010: Workshop on Invariant Generation 2010*, ser. EPiC Series in Computing, A. Voronkov, L. Kovacs, and N. Björner, Eds., vol. 1. EasyChair, 2012, pp. 44–58. [Online]. Available: <https://doi.org/10.29007/npn7>
- [7] C. Daws and S. Yovine, “Reducing the number of clock variables of timed automata,” in *Proceedings of the 17th IEEE Real-Time Systems Symposium*. IEEE Computer Society, 1996, pp. 73–81. [Online]. Available: <https://doi.org/10.1109/REAL.1996.563702>
- [8] S. Tripakis, “Checking timed Büchi automata emptiness on simulation graphs,” *ACM Trans. Comput. Logic*, vol. 10, no. 3, Apr. 2009. [Online]. Available: <https://doi.org/10.1145/1507244.1507245>
- [9] F. Herbreteau and G. Point, “TChecker: model checker for timed automata,” <https://github.com/ticktac-project/tchecker>, 2025, version 0.8.
- [10] A. Schimpf, S. Merz, and J.-G. Smaus, “Construction of Büchi automata for LTL model checking verified in Isabelle/HOL,” in *Theorem Proving in Higher Order Logics (TPHOLs 2009)*, ser. Lecture Notes in Computer Science, vol. 5674. Springer, 2009, pp. 424–439.
- [11] J. Esparza, P. Lammich, R. Neumann, T. Nipkow, A. Schimpf, and J.-G. Smaus, “A fully verified executable LTL model checker,” in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 463–478.
- [12] S. Wimmer and P. Lammich, “Verified model checking of timed automata,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2018)*, ser. Lecture Notes in Computer Science. Springer, 2018, pp. 61–78.
- [13] S. Wimmer, “Munta: A verified model checker for timed automata,” in *Formal Modeling and Analysis of Timed Systems (FORMATS 2019)*, ser. Lecture Notes in Computer Science. Springer, 2019, pp. 236–243.
- [14] P. Letouzey, “Extraction in Coq: An overview,” in *Logic and Theory of Algorithms, Fourth Conference on Computability in Europe (CiE 2008)*, ser. Lecture Notes in Computer Science, vol. 5028. Springer, 2008, pp. 359–369.

...